

Amazon EC2 – Basics



Amazon EC2

- EC2 is one of the most popular of AWS' offering
- EC2 = Elastic Compute Cloud = Infrastructure as a Service
- It mainly consists in the capability of :
 - Renting virtual machines (EC2)
 - Storing data on virtual drives (EBS)
 - Distributing load across machines (ELB)
 - Scaling the services using an auto-scaling group (ASG)
- Knowing EC2 is fundamental to understand how the Cloud works

EC2 sizing & configuration options

- Operating System (OS): Linux, Windows or Mac OS
- How much compute power & cores (CPU)
- How much random-access memory (RAM)
- How much storage space:
 - Network-attached (EBS & EFS)
 - hardware (EC2 Instance Store)
- Network card: speed of the card, Public IP address
- Firewall rules: security group
- Bootstrap script (configure at first launch): EC2 User Data

EC2 User Data

- It is possible to bootstrap our instances using an [EC2 User data](#) script.
- [bootstrapping](#) means launching commands when a machine starts
- That script is [only run once](#) at the instance [first start](#)
- EC2 user data is used to automate boot tasks such as:
 - Installing updates
 - Installing software
 - Downloading common files from the internet
 - Anything you can think of
- The EC2 User Data Script runs with the root user

EC2 User Data

User data - *optional* | [Info](#)

Upload a file with your user data or enter it in the field.

 [Choose file](#)

```
#!/bin/bash
apt-get update -y
apt-get upgrade -y
curl -fsSL https://deb.nodesource.com/setup_lts.x | bash -
apt-get install -y nodejs nginx
npm install -g pm2
systemctl enable nginx
systemctl start nginx
```

☐ User data has already been base64 encoded

EC2 Instance Types - Overview

- You can use different types of EC2 instances that are optimised for different use cases (<https://aws.amazon.com/ec2/instance-types/>)
- AWS has the following naming convention:

m5.2xlarge

- **m**: instance class
- **5**: generation (AWS improves them over time)
- **2xlarge**: size within the instance class

General Purpose

Compute Optimized

Memory Optimized

Accelerated Computing

Storage Optimized

HPC Optimized

Instance Features

Measuring Instance
Performance

EC2 Instance Types – General Purpose

- Great for a diversity of workloads such as web servers or code repositories
- Balance between:
 - Compute
 - Memory
 - Networking
- In the course, we will be using the t2.micro which is a General Purpose EC2 instance

General Purpose

General purpose instances provide a balance of compute, memory and networking resources, and can be used for a variety of diverse workloads. These instances are ideal for applications that use these resources in equal proportions such as web servers and code repositories.

Mac	T4g	T3	T3a	T2	M6g	M5	M5a	M5n	M5zn	M4	A1
-----	-----	----	-----	----	-----	----	-----	-----	------	----	----

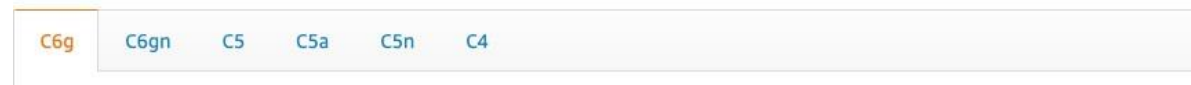
** this list will evolve over time, please check the AWS website for the latest information*

EC2 Instance Types – Compute Optimized

- Great for compute-intensive tasks that require high performance processors:
 - Batch processing workloads
 - Media transcoding
 - High performance web servers
 - High performance computing (HPC)
 - Scientific modeling & machine learning
 - Dedicated gaming servers

Compute Optimized

Compute Optimized instances are ideal for compute bound applications that benefit from high performance processors. Instances belonging to this family are well suited for batch processing workloads, media transcoding, high performance web servers, high performance computing (HPC), scientific modeling, dedicated gaming servers and ad server engines, machine learning inference and other compute intensive applications.



** this list will evolve over time, please check the AWS website for the latest information*

EC2 Instance Types – Memory Optimized

- Fast performance for workloads that process large data sets in memory
- Use cases:
 - High performance, relational/non-relational databases
 - Distributed web scale cache stores
 - In-memory databases optimized for BI (business intelligence)
 - Applications performing real-time processing of big unstructured data

Memory Optimized

Memory optimized instances are designed to deliver fast performance for workloads that process large data sets in memory.

R6g	R5	R5a	R5b	R5n	R4	X1e	X1	High Memory	z1d
-----	----	-----	-----	-----	----	-----	----	-------------	-----

** this list will evolve over time, please check the AWS website for the latest information*

EC2 Instance Types – Storage Optimized

- Great for storage-intensive tasks that require high, sequential read and write access to large data sets on local storage
- Use cases:
 - High frequency online transaction processing (OLTP) systems
 - Relational & NoSQL databases
 - Cache for in-memory databases (for example, Redis)
 - Data warehousing applications
 - Distributed file systems

Storage Optimized

Storage optimized instances are designed for workloads that require high, sequential read and write access to very large data sets on local storage. They are optimized to deliver tens of thousands of low-latency, random I/O operations per second (IOPS) to applications.

I3	I3en	D2	D3	D3en	H1
----	------	----	----	------	----

** this list will evolve over time, please check the AWS website for the latest information*

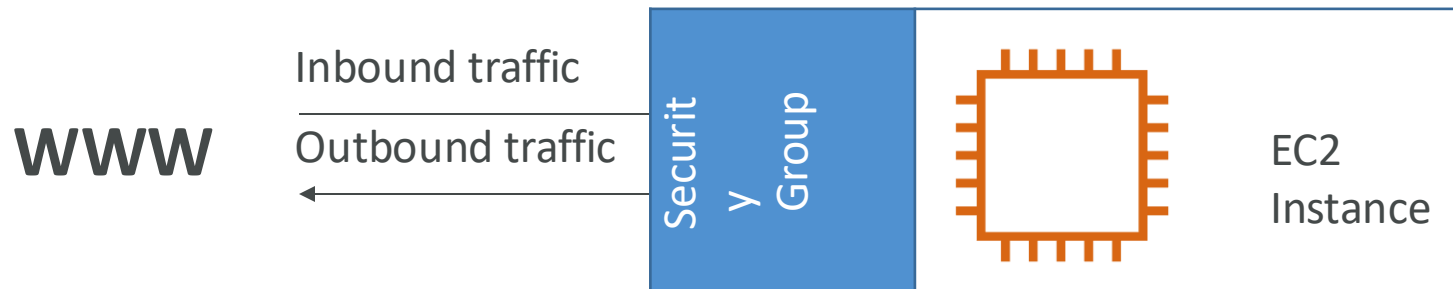
EC2 Instance Types: example

Instance	vCPU	Mem (GiB)	Storage	Network Performance	EBS Bandwidth (Mbps)
t2.micro	1	1	EBS-Only	Low to Moderate	
t2.xlarge	4	16	EBS-Only	Moderate	
c5d.4xlarge	16	32	1 x 400 NVMe SSD	Up to 10 Gbps	4,750
r5.16xlarge	64	512	EBS Only	20 Gbps	13,600
m5.8xlarge	32	128	EBS Only	10 Gbps	6,800

Great website: <https://instances.vantage.sh>

Introduction to Security Groups

- Security Groups are the fundamental of network security in AWS
- They control how traffic is allowed into or out of our EC2 Instances.



- Security groups only contain **allow** rules
- Security groups rules can reference by IP or by security group

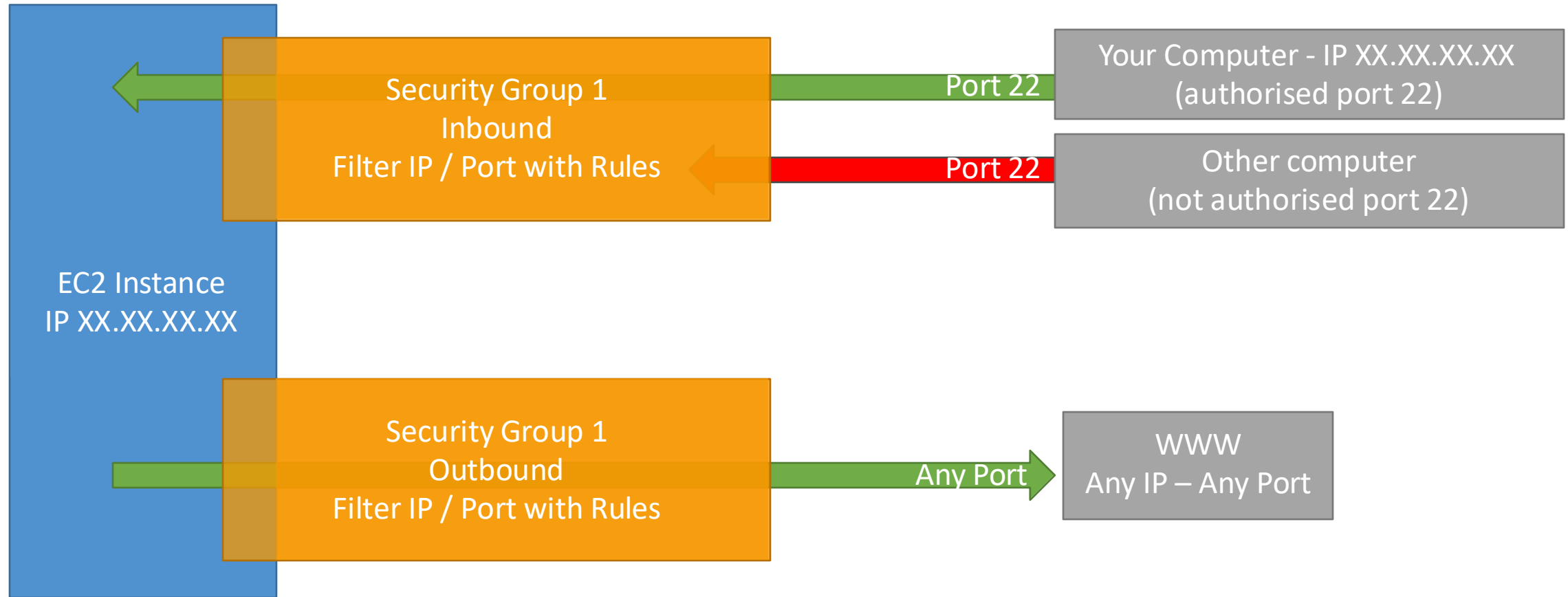
Security Groups

Deeper Dive

- Security groups are acting as a “firewall” on EC2 instances
- They regulate:
 - Access to Ports
 - Authorised IP ranges - IPv4 and IPv6
 - Control of inbound network (from other to the instance)
 - Control of outbound network (from the instance to other)

Type ⓘ	Protocol ⓘ	Port Range ⓘ	Source ⓘ	Description ⓘ
HTTP	TCP	80	0.0.0.0/0	test http page
SSH	TCP	22	122.149.196.85/32	
Custom TCP Rule	TCP	4567	0.0.0.0/0	java app

Security Groups Diagram



Security Groups

Good to know

- Can be attached to multiple instances
- Locked down to a region / VPC combination
- Does live “outside” the EC2 - if traffic is blocked the EC2 instance won’t see it
- It’s good to maintain one separate security group for SSH access
- If your application is not accessible (time out), then it’s a security group issue
- If your application gives a “connection refused” error, then it’s an application error or it’s not launched
- All inbound traffic is **blocked** by default
- All outbound traffic is **authorised** by default

Classic Ports to know

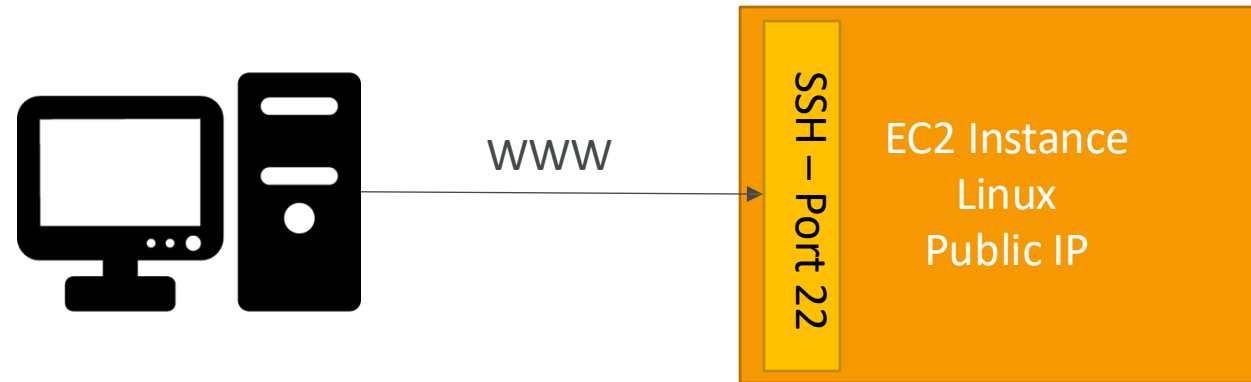
- 22 = SSH (Secure Shell) - log into a Linux instance
- 21 = FTP (File Transfer Protocol) - upload files into a file share
- 22 = SFTP (Secure File Transfer Protocol) - upload files using SSH
- 80 = HTTP - access unsecured websites
- 443 = HTTPS - access secured websites
- 3389 = RDP (Remote Desktop Protocol) - log into a Windows instance

SSH troubleshooting

- Students have the most problems with SSH
- If things don't work...
 1. Re-watch the lecture. You may have missed something
 2. Read the troubleshooting guide
 3. Try EC2 Instance Connect
- If one method works (SSH, Putty or EC2 Instance Connect) you're good
- If no method works, that's okay, the course won't use SSH much

How to SSH into your EC2 Instance [Linux](#) [/ Mac OS X](#)

- We'll learn how to SSH into your EC2 instance using [Linux / Mac](#)
- SSH is one of the most important function. It allows you to control a remote machine, all using the command line.



- We will see how we can configure OpenSSH [~/ssh/config](#) to facilitate the SSH into our EC2 instances

EC2 Instance Connect

- Connect to your EC2 instance within your browser
- No need to use your key file that was downloaded
- The “magic” is that a temporary key is uploaded onto EC2 by AWS
- Works only out-of-the-box with Amazon Linux 2
- Need to make sure the port 22 is still opened!

EC2 Instances Purchasing Options

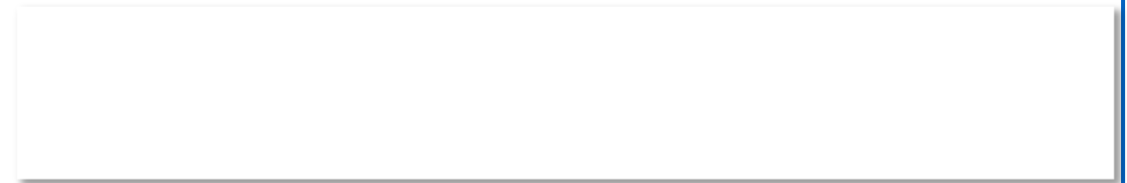
- On-Demand Instances - short workload, predictable pricing, pay by second
- Reserved (1 & 3 years)
 - Reserved Instances - long workloads
 - Convertible Reserved Instances - long workloads with flexible instances
- Savings Plans (1 & 3 years) - commitment to an amount of usage, long workload
- Spot Instances - short workloads, cheap, can lose instances (less reliable)
- Dedicated Hosts - book an entire physical server, control instance placement
- Dedicated Instances - no other customers will share your hardware
- Capacity Reservations - reserve capacity in a specific AZ for any duration

EC2 On Demand

- Pay for what you use:
 - Linux or Windows - billing per second, after the first minute
 - All other operating systems - billing per hour
- Has the highest cost but no upfront payment
- No long-term commitment
- Recommended for short-term and un-interrupted workloads, where you can't predict how the application will behave

EC2 Reserved Instances

- Up to 72% discount compared to On-demand
- You reserve a specific instance attributes (Instance Type, Region, Tenancy, OS)
- Reservation Period - 1 year (+discount) or 3 years (+++discount)
- Payment Options - No Upfront (+), Partial Upfront (++), All Upfront (+++)
- Reserved Instance's Scope - Regional or Zonal (reserve capacity in an AZ)
- Recommended for steady-state usage applications (think database)
- You can buy and sell in the Reserved Instance Marketplace
- Convertible Reserved Instance
 - Can change the EC2 instance type, instance family, OS, scope and tenancy
 - Up to 66% discount



EC2 Savings Plans

- Get a discount based on long-term usage (up to 72% - same as RIs)
- Commit to a certain type of usage (\$10/hour for 1 or 3 years)
- Usage beyond EC2 Savings Plans is billed at the On-Demand price
- Locked to a specific instance family & AWS region (e.g., M5 in us-east-1)
- Flexible across:
 - Instance Size (e.g., m5.xlarge, m5.2xlarge)
 - OS (e.g., Linux, Windows)
 - Tenancy (Host, Dedicated, Default)



EC2 Spot Instances

- Can get a discount of up to 90% compared to On-demand
- Instances that you can “lose” at any point of time if your max price is less than the current spot price
- The MOST cost-efficient instances in AWS
- Useful for workloads that are resilient to failure
 - Batch jobs
 - Data analysis
 - Image processing
 - Any distributed workloads
 - Workloads with a flexible start and end time
- Not suitable for critical jobs or databases

EC2 Dedicated Hosts

- A physical server with EC2 instance capacity fully dedicated to your use
- Allows you address compliance requirements and use your existing server-bound software licenses (per-socket, per-core, per-VM software licenses)
- Purchasing Options:
 - On-demand - pay per second for active Dedicated Host
 - Reserved - 1 or 3 years (No Upfront, Partial Upfront, All Upfront)
- The most expensive option
- Useful for software that have complicated licensing model (BYOL - Bring Your Own License)
- Or for companies that have strong regulatory or compliance needs

EC2 Dedicated Instances

- Instances run on hardware that's dedicated to you
- May share hardware with other instances in same account
- No control over instance placement (can move hardware after Stop / Start)

Characteristic	Dedicated Instances	Dedicated Hosts
Enables the use of dedicated physical servers	X	X
Per instance billing (subject to a \$2 per region fee)	X	
Per host billing		X
Visibility of sockets, cores, host ID		X
Affinity between a host and instance		X
Targeted instance placement		X
Automatic instance placement	X	X
Add capacity using an allocation request		X

EC2 Capacity Reservations

- Reserve On-Demand instances capacity in a specific AZ for any duration
- You always have access to EC2 capacity when you need it
- No time commitment (create/cancel anytime), no billing discounts
- Combine with Regional Reserved Instances and Savings Plans to benefit from billing discounts
- You're charged at On-Demand rate whether you run instances or not
- Suitable for short-term, uninterrupted workloads that needs to be in a specific AZ

Which purchasing option is right for me?



- On demand: coming and staying in resort whenever we like, we pay the full price
- Reserved: like planning ahead and if we plan to stay for a long time, we may get a good discount.
- Savings Plans: pay a certain amount per hour for certain period and stay in any room type (e.g., King, Suite, Sea View, ...)
- Spot instances: the hotel allows people to bid for the empty rooms and the highest bidder keeps the rooms. You can get kicked out at any time
- Dedicated Hosts: We book an entire building of the resort
- Capacity Reservations: you book a room for a period with full price even you don't stay in it

Price Comparison

Example – m4.large – us-east-1

Price Type	Price (per hour)
On-Demand	\$0.10
Spot Instance (Spot Price)	\$0.038 - \$0.039 (up to 61% off)
Reserved Instance (1 year)	\$0.062 (No Upfront) - \$0.058 (All Upfront)
Reserved Instance (3 years)	\$0.043 (No Upfront) - \$0.037 (All Upfront)
EC2 Savings Plan (1 year)	\$0.062 (No Upfront) - \$0.058 (All Upfront)
Reserved Convertible Instance (1 year)	\$0.071 (No Upfront) - \$0.066 (All Upfront)
Dedicated Host	On-Demand Price
Dedicated Host Reservation	Up to 70% off
Capacity Reservations	On-Demand Price

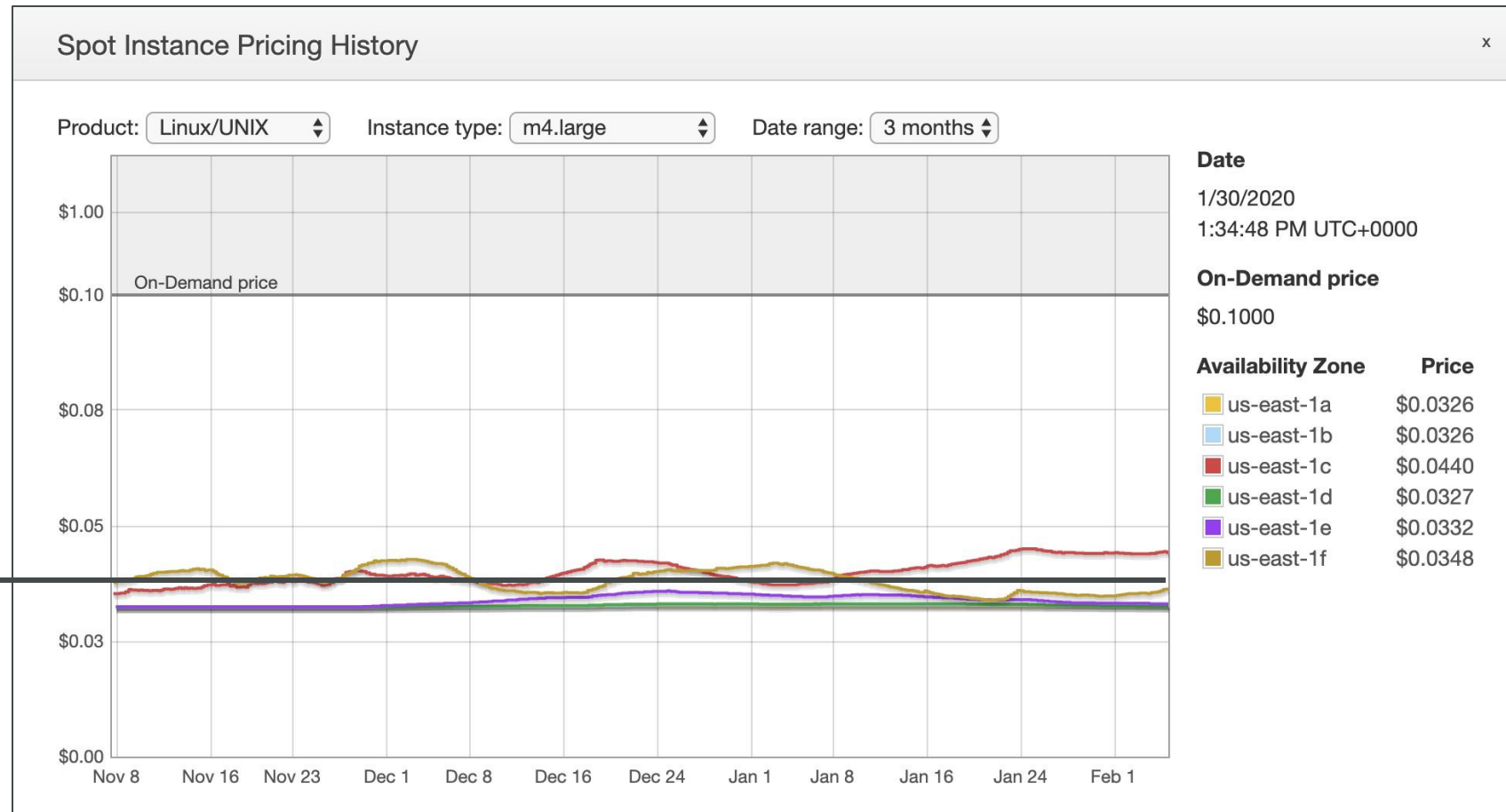


EC2 Spot Instance Requests

- Can get a discount of up to 90% compared to On-demand
- Define max spot price and get the instance while current spot price < max
 - The hourly spot price varies based on offer and capacity
 - If the current spot price > your max price you can choose to stop or terminate your instance with a 2 minutes grace period.
- Other strategy: Spot Block
 - “block” spot instance during a specified time frame (1 to 6 hours) without interruptions
 - In rare situations, the instance may be reclaimed
- Used for batch jobs, data analysis, or workloads that are resilient to failures.
- Not great for critical jobs or databases

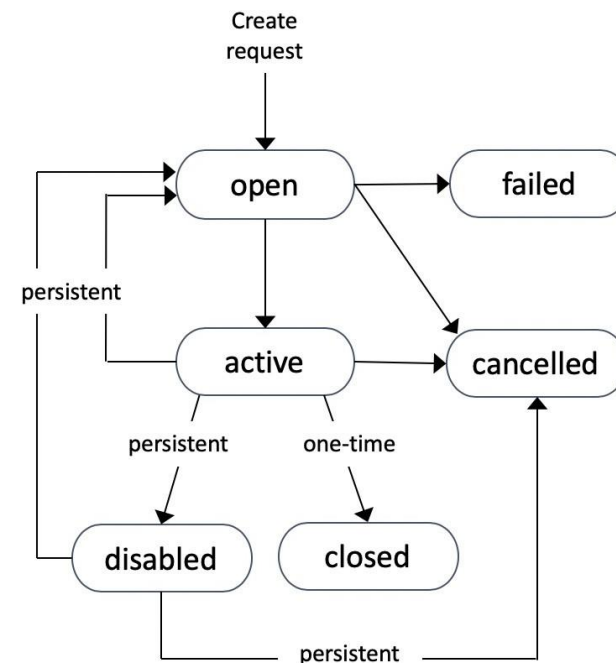
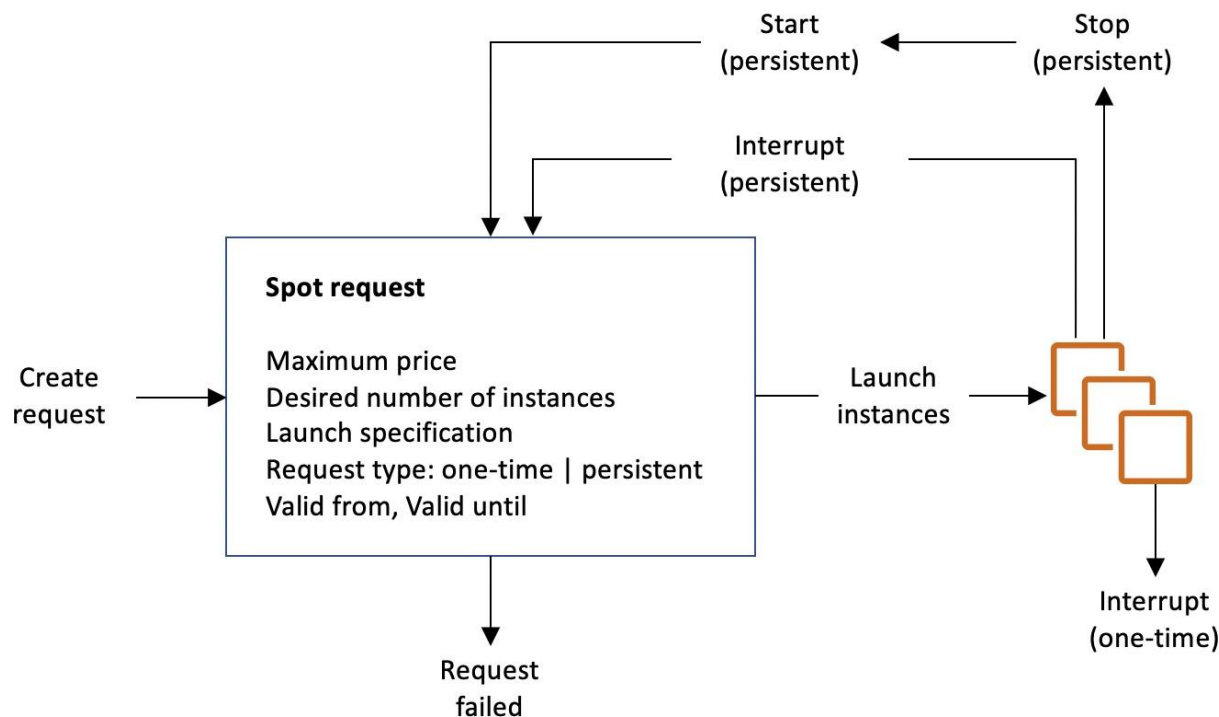
EC2 Spot Instances Pricing

User-defined max price



<https://console.aws.amazon.com/ec2sp/v1/spot/home?region=us-east-1#>

How to terminate Spot Instances?



You can only cancel Spot Instance requests that are **open, active, or disabled**.

Cancelling a Spot Request does not terminate instances

You must first cancel a Spot Request, and then terminate the associated Spot Instances

Spot Fleets

- Spot Fleets = set of Spot Instances + (optional) On-Demand Instances
- The Spot Fleet will try to meet the target capacity with price constraints
 - Define possible launch pools: instance type (m5.large), OS, Availability Zone
 - Can have multiple launch pools, so that the fleet can choose
 - Spot Fleet stops launching instances when reaching capacity or max cost
- Strategies to allocate Spot Instances:
 - lowestPrice: from the pool with the lowest price (cost optimization, short workload)
 - diversified: distributed across all pools (great for availability, long workloads)
 - capacityOptimized: pool with the optimal capacity for the number of instances
 - priceCapacityOptimized (recommended): pools with highest capacity available, then select the pool with the lowest price (best choice for most workloads)
- Spot Fleets allow us to automatically request Spot Instances with the lowest price

Amazon EC2 – Associate

Private vs Public IP (IPv4)

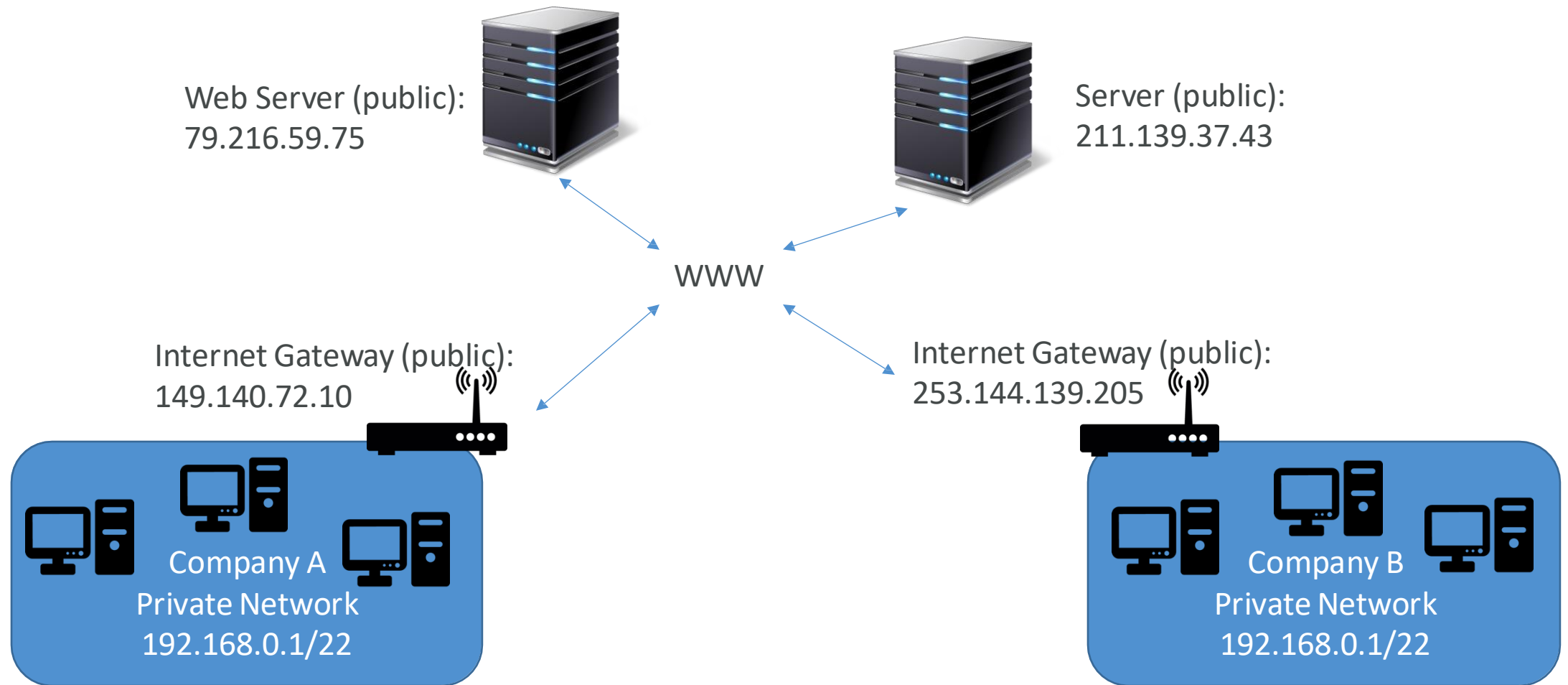
- Networking has two sorts of IPs. IPv4 and IPv6:
 - IPv4: **1.160.10.240**
 - IPv6: **3ffe:1900:4545:3:200:f8ff:fe21:67cf**
- In this course, we will only be using IPv4.
- IPv4 is still the most common format used online.
- IPv6 is newer and solves problems for the Internet of Things (IoT).
- IPv4 allows for **3.7 billion** different addresses in the public space
- IPv4: [0-255].[0-255].[0-255].[0-255].

Private vs Public IP (IPv4)

Fundamental Differences

- Public IP:
 - Public IP means the machine can be identified on the internet (WWW)
 - Must be unique across the whole web (not two machines can have the same public IP).
 - Can be geo-located easily
- Private IP:
 - Private IP means the machine can only be identified on a private network only
 - The IP must be unique across the private network
 - BUT two different private networks (two companies) can have the same IPs.
 - Machines connect to WWW using a NAT + internet gateway (a proxy)
 - Only a specified range of IPs can be used as private IP

Private vs Public IP (IPv4) Example



Elastic IPs

- When you stop and then start an EC2 instance, it can change its public IP.
- If you need to have a fixed public IP for your instance, you need an Elastic IP
- An Elastic IP is a public IPv4 IP you own as long as you don't delete it
- You can attach it to one instance at a time

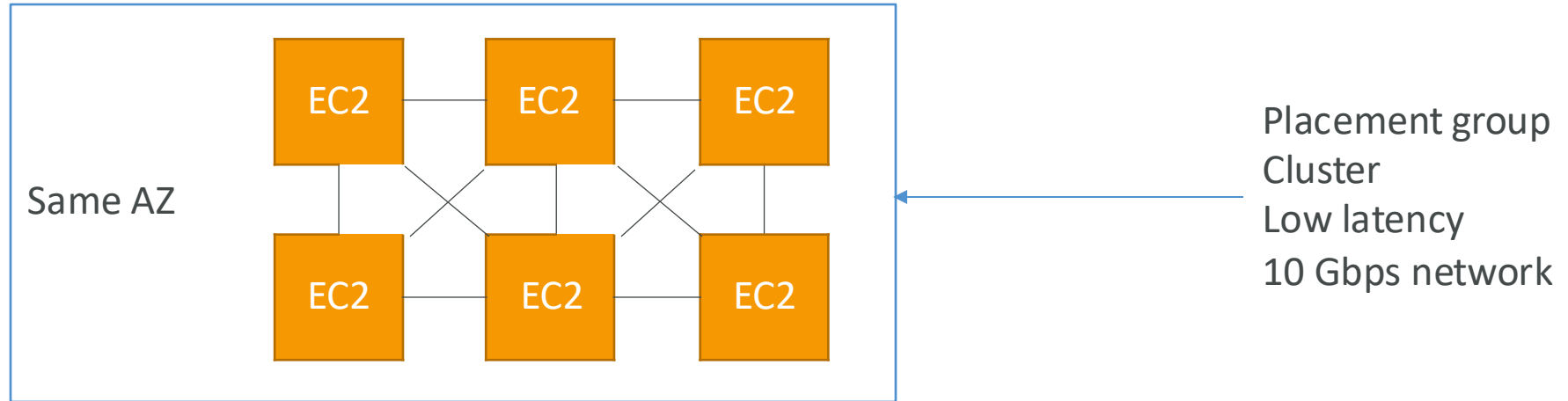
Elastic IP

- With an Elastic IP address, you can mask the failure of an instance or software by rapidly remapping the address to another instance in your account.
- You can only have 5 Elastic IP in your account (you can ask AWS to increase that).
- Overall, **try to avoid using Elastic IP:**
 - They often reflect poor architectural decisions
 - Instead, use a random public IP and register a DNS name to it
 - Or, as we'll see later, use a Load Balancer and don't use a public IP

Placement Groups

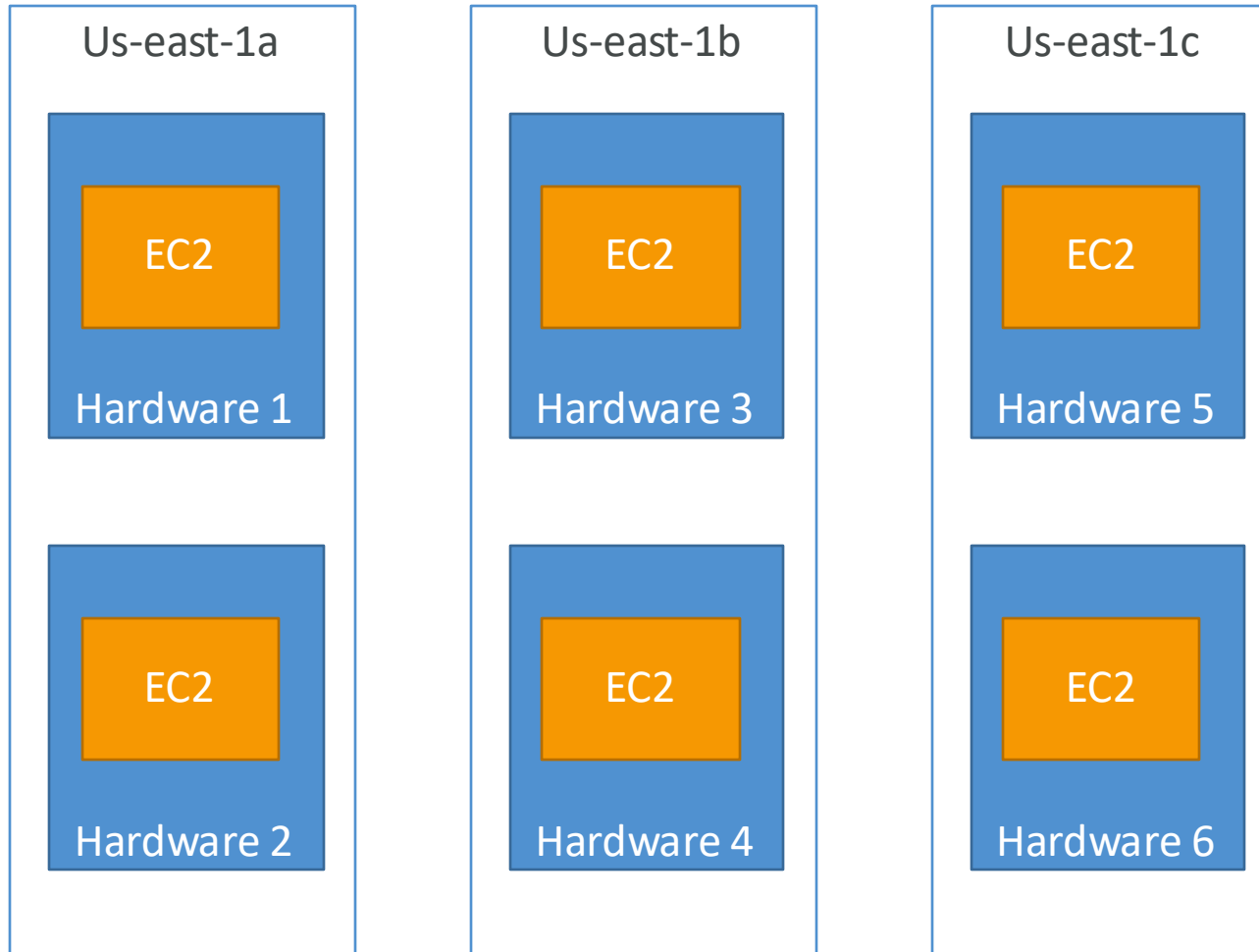
- Sometimes you want control over the EC2 Instance placement strategy
- That strategy can be defined using placement groups
- When you create a placement group, you specify one of the following strategies for the group:
 - *Cluster*—clusters instances into a low-latency group in a single Availability Zone
 - *Spread*—spreads instances across underlying hardware (max 7 instances per group per AZ)
 - *Partition*—spreads instances across many different partitions (which rely on different sets of racks) within an AZ. Scales to 100s of EC2 instances per group (Hadoop, Cassandra, Kafka)

Placement Groups Cluster



- Pros: Great network (10 Gbps bandwidth between instances with Enhanced Networking enabled - recommended)
- Cons: If the AZ fails, all instances fail at the same time
- Use case:
 - Big Data job that needs to complete fast
 - Application that needs extremely low latency and high network throughput

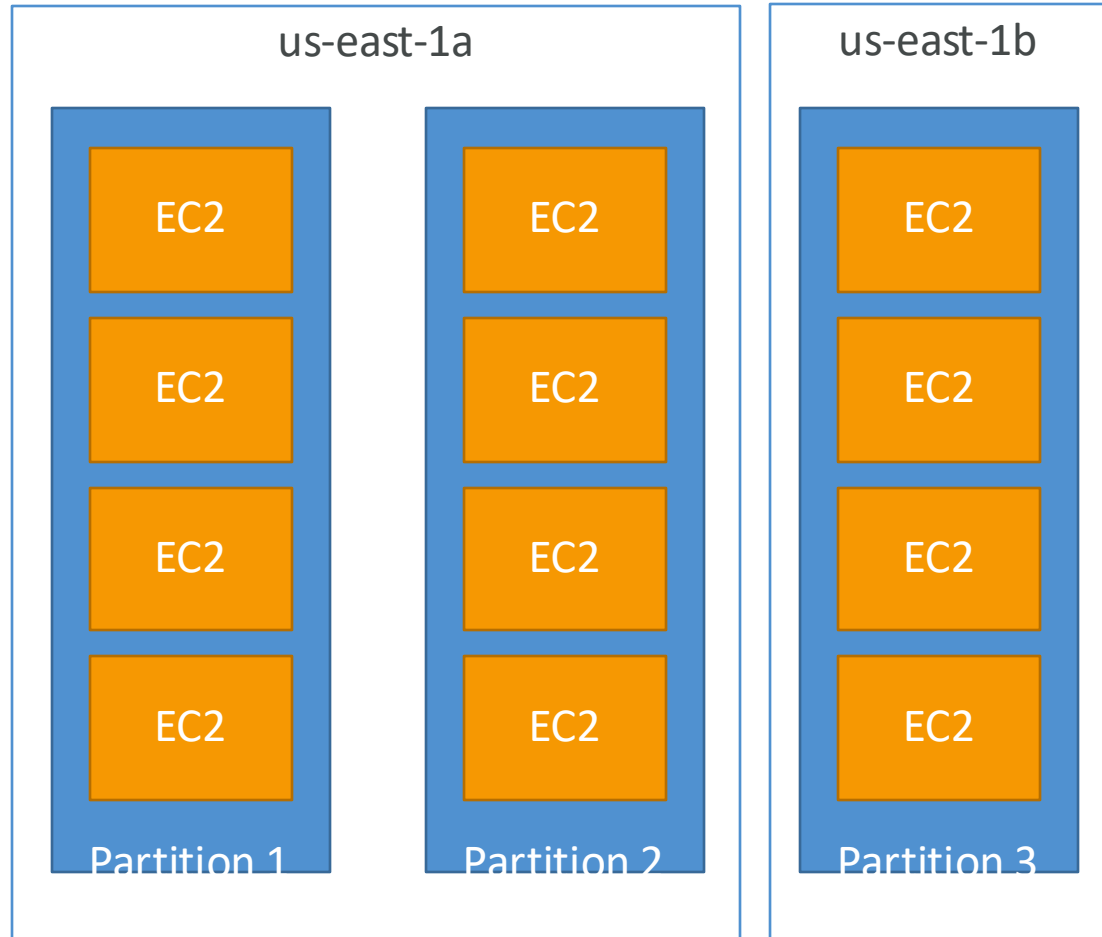
Placement Groups Spread



- **Pros:**
 - Can span across Availability Zones (AZ)
 - Reduced risk is simultaneous failure
 - EC2 Instances are on different physical hardware
- **Cons:**
 - Limited to 7 instances per AZ per placement group
- **Use case:**
 - Application that needs to maximize high availability
 - Critical Applications where each instance must be isolated from failure from each other

Placements Groups

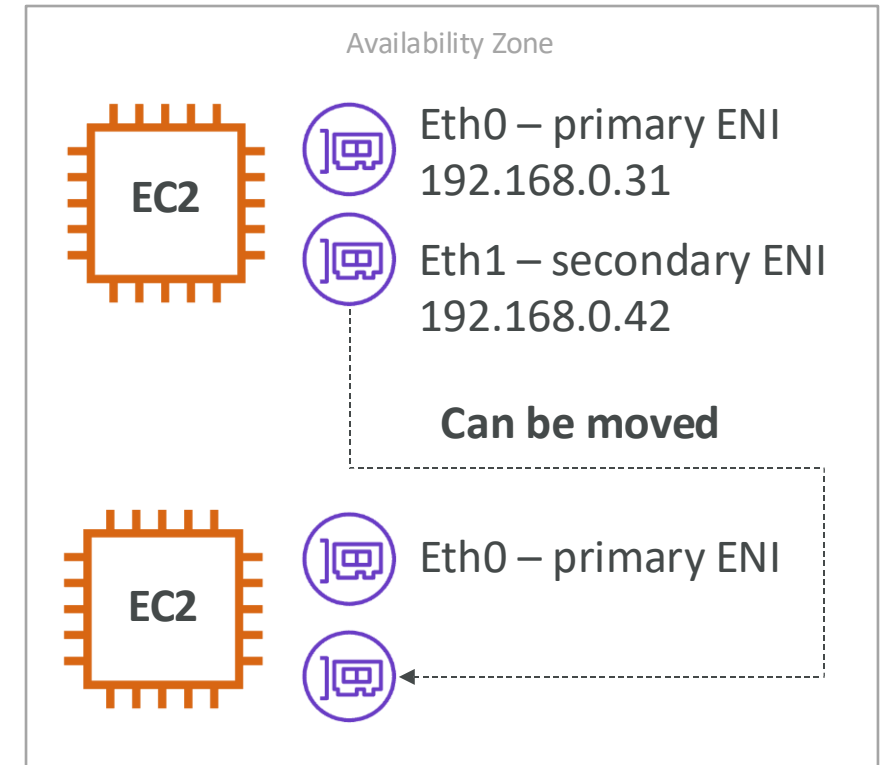
Partition



- Up to 7 partitions per AZ
- Can span across multiple AZs in the same region
- Up to 100s of EC2 instances
- The instances in a partition do not share racks with the instances in the other partitions
- A partition failure can affect many EC2 but won't affect other partitions
- EC2 instances get access to the partition information as metadata
- Use cases: HDFS, HBase, Cassandra, Kafka

Elastic Network Interfaces (ENI)

- Logical component in a VPC that represents a virtual network card
- The ENI can have the following attributes:
 - Primary private IPv4, one or more secondary IPv4
 - One Elastic IP (IPv4) per private IPv4
 - One Public IPv4
 - One or more security groups
 - A MAC address
- You can create ENI independently and attach them on the fly (move them) on EC2 instances for failover
- Bound to a specific availability zone (AZ)

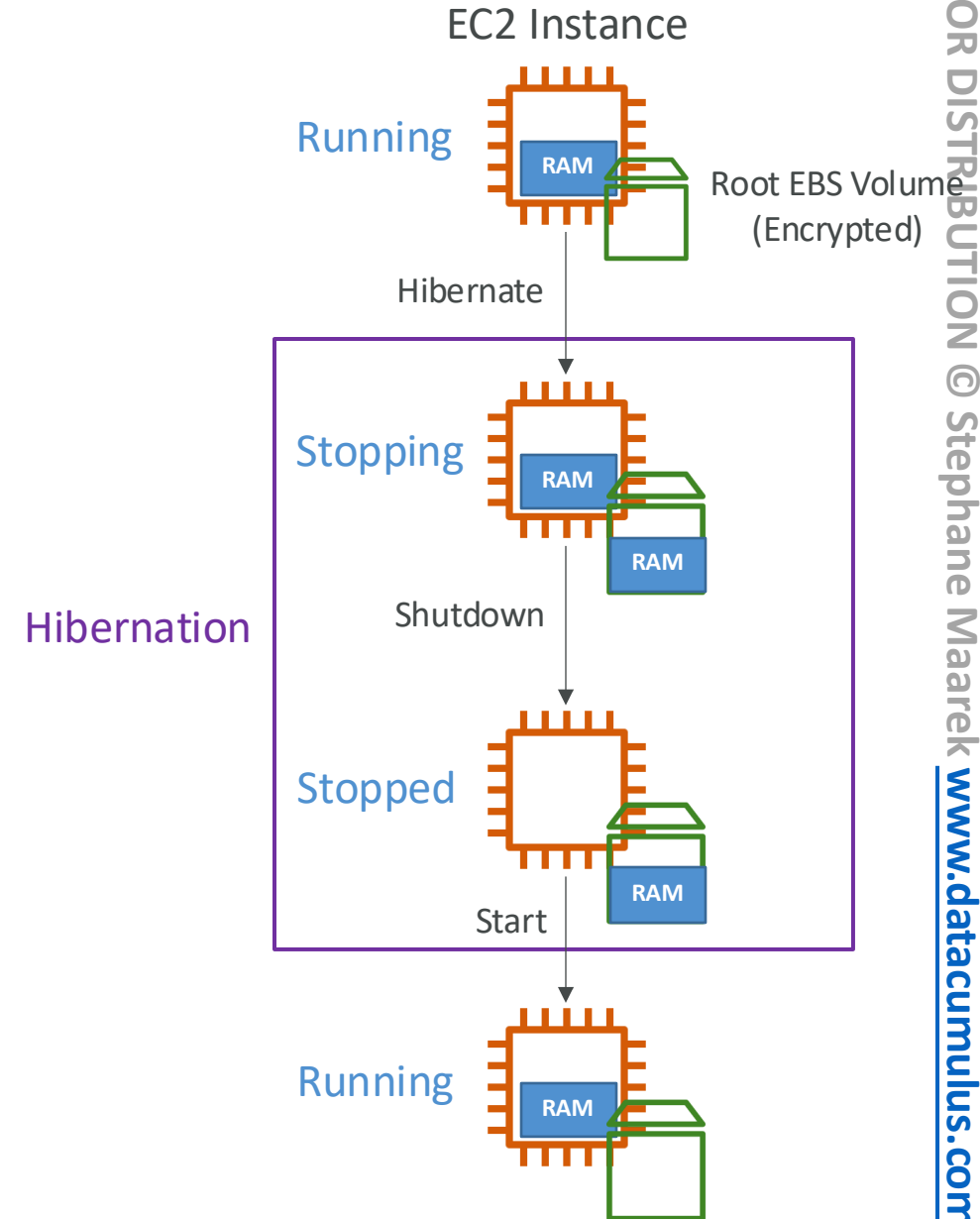


EC2 Hibernate

- We know we can stop, terminate instances
 - Stop - the data on disk (EBS) is kept intact in the next start
 - Terminate - any EBS volumes (root) also set-up to be destroyed is lost
- On start, the following happens:
 - First start: the OS boots & the EC2 User Data script is run
 - Following starts: the OS boots up
 - Then your application starts, caches get warmed up, and that can take time!

EC2 Hibernate

- Introducing EC2 Hibernate:
 - The in-memory (RAM) state is preserved
 - The instance boot is much faster!
(the OS is not stopped / restarted)
 - Under the hood: the RAM state is written to a file in the root EBS volume
 - The root EBS volume must be encrypted
- Use cases:
 - Long-running processing
 - Saving the RAM state
 - Services that take time to initialize



EC2 Hibernate – Good to know

- Supported Instance Families - C3, C4, C5, I3, M3, M4, R3, R4, T2, T3, ...
- Instance RAM Size - must be less than 150 GB.
- Instance Size - not supported for bare metal instances.
- AMI - Amazon Linux 2, Linux AMI, Ubuntu, RHEL, CentOS & Windows...
- Root Volume - must be EBS, encrypted, not instance store, and large
- Available for On-Demand, Reserved and Spot Instances

- An instance can NOT be hibernated more than 60 days

High Availability & Scalability

Scalability & High Availability

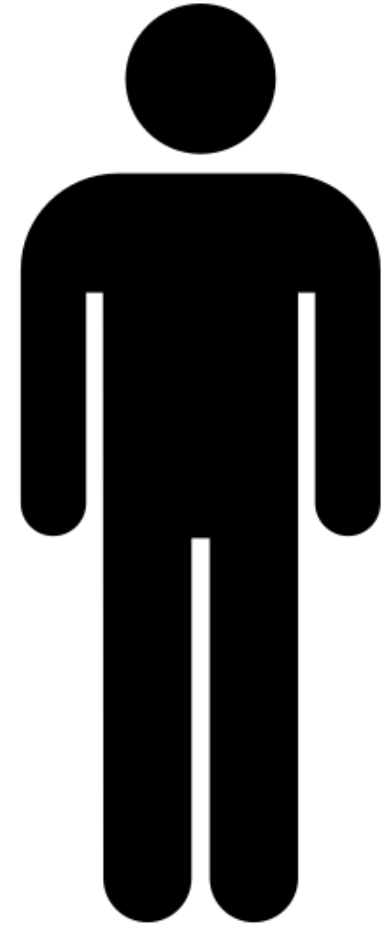
- Scalability means that an application / system can handle greater loads by adapting.
- There are two kinds of scalability:
 - Vertical Scalability
 - Horizontal Scalability (= elasticity)
- Scalability is linked but different to High Availability
- Let's deep dive into the distinction, using a call center as an example

Vertical Scalability

- Vertically scalability means increasing the size of the instance
- For example, your application runs on a t2.micro
- Scaling that application vertically means running it on a t2.large
- Vertical scalability is very common for non distributed systems, such as a database.
- RDS, ElastiCache are services that can scale vertically.
- There's usually a limit to how much you can vertically scale (hardware limit)



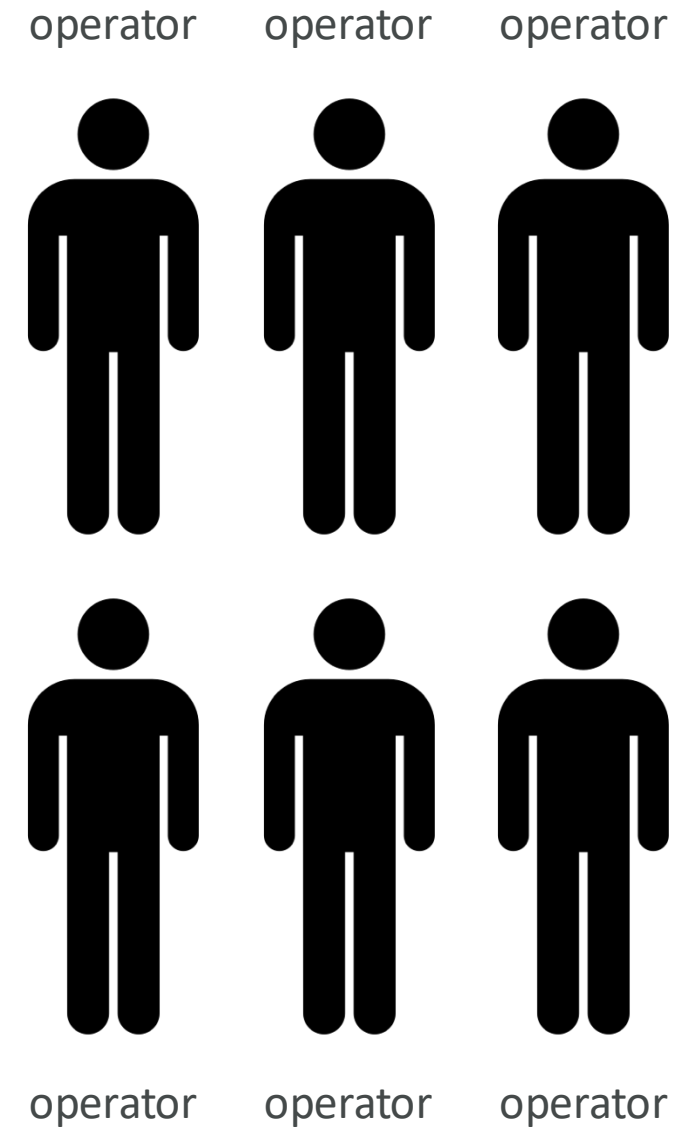
junior operator



senior operator

Horizontal Scalability

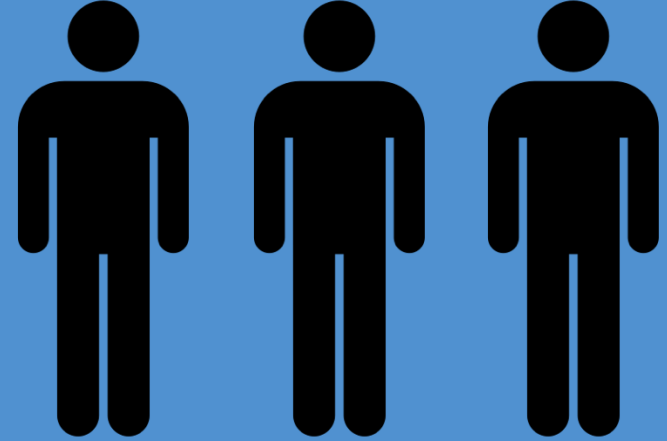
- Horizontal Scalability means increasing the number of instances / systems for your application
- Horizontal scaling implies distributed systems.
- This is very common for web applications / modern applications
- It's easy to horizontally scale thanks the cloud offerings such as Amazon EC2



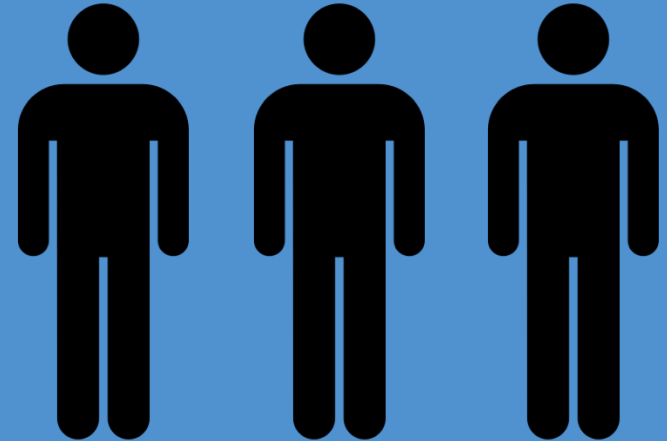
High Availability

- High Availability usually goes hand in hand with horizontal scaling
- High availability means running your application / system in at least 2 data centers (== Availability Zones)
- The goal of high availability is to survive a data center loss
- The high availability can be passive (for RDS Multi AZ for example)
- The high availability can be active (for horizontal scaling)

first building in New York



second building in San Francisco

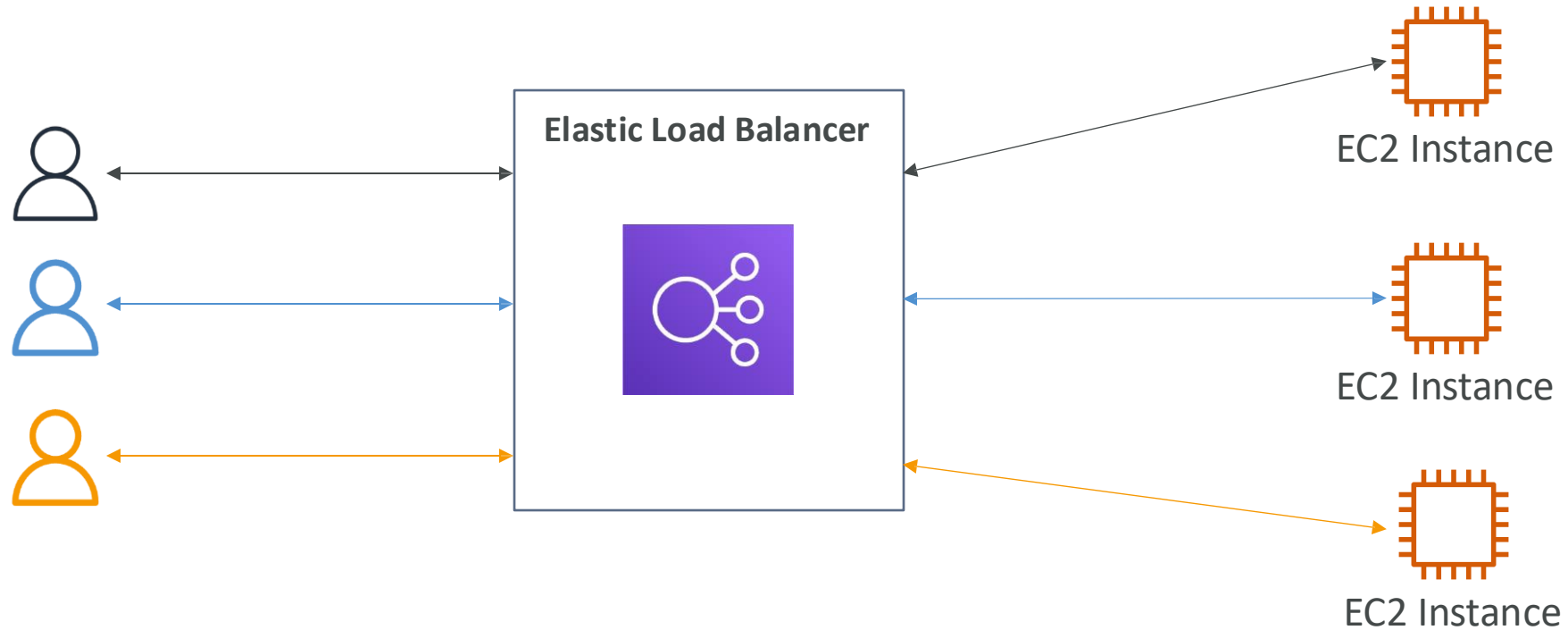


High Availability & Scalability For EC2

- Vertical Scaling: Increase instance size (= scale up / down)
 - From: t2.nano - 0.5G of RAM, 1 vCPU
 - To: u-12tb1.metal - 12.3 TB of RAM, 448 vCPUs
- Horizontal Scaling: Increase number of instances (= scale out / in)
 - Auto Scaling Group
 - Load Balancer
- High Availability: Run instances for the same application across multi AZ
 - Auto Scaling Group multi AZ
 - Load Balancer multi AZ

What is load balancing?

- Load Balancers are servers that forward traffic to multiple servers (e.g., EC2 instances) downstream



Why use a load balancer?

- Spread load across multiple downstream instances
- Expose a single point of access (DNS) to your application
- Seamlessly handle failures of downstream instances
- Do regular health checks to your instances
- Provide SSL termination (HTTPS) for your websites
- Enforce stickiness with cookies
- High availability across zones
- Separate public traffic from private traffic

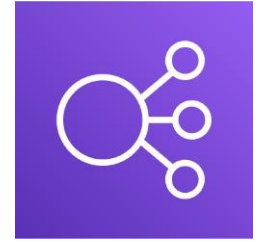
Why use an Elastic Load Balancer?

- An Elastic Load Balancer is a **managed load balancer**
 - AWS guarantees that it will be working
 - AWS takes care of upgrades, maintenance, high availability
 - AWS provides only a few configuration knobs
- It costs less to setup your own load balancer but it will be a lot more effort on your end
- It is integrated with many AWS offerings / services
 - EC2, EC2 Auto Scaling Groups, Amazon ECS
 - AWS Certificate Manager (ACM), CloudWatch
 - Route 53, AWS WAF, AWS Global Accelerator

Health Checks

- Health Checks are crucial for Load Balancers
- They enable the load balancer to know if instances it forwards traffic to are available to reply to requests
- The health check is done on a port and a route (/health is common)
- If the response is not 200 (OK), then the instance is unhealthy





Types of load balancer on AWS

- AWS has 4 kinds of managed Load Balancers
- Classic Load Balancer (v1 - old generation) - 2009 - CLB
 - HTTP, HTTPS, TCP, SSL (secure TCP)
- Application Load Balancer (v2 - new generation) - 2016 - ALB
 - HTTP, HTTPS, WebSocket
- Network Load Balancer (v2 - new generation) - 2017 - NLB
 - TCP, TLS (secure TCP), UDP
- Gateway Load Balancer - 2020 - GWLB
 - Operates at layer 3 (Network layer) - IP Protocol
- Overall, it is recommended to use the newer generation load balancers as they provide more features
- Some load balancers can be setup as **internal** (private) or **external** (public) ELBs

Load Balancer Security Groups



Load Balancer Security Group:

Type ⓘ	Protocol ⓘ	Port Range ⓘ	Source ⓘ	Description ⓘ
HTTP	TCP	80	0.0.0.0/0	Allow HTTP from an...
HTTPS	TCP	443	0.0.0.0/0	Allow HTTPS from a...

Application Security Group: Allow traffic only from Load Balancer

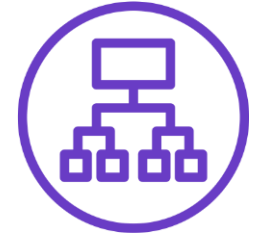
Type ⓘ	Protocol ⓘ	Port Range ⓘ	Source ⓘ	Description ⓘ
HTTP	TCP	80	sg-054b5ff5ea02f2b6e (load-b	Allow Traffic only...



Application Load Balancer (v2)

- Application load balancers is Layer 7 (HTTP)
- Load balancing to multiple HTTP applications across machines (target groups)
- Load balancing to multiple applications on the same machine (ex: containers)
- Support for HTTP/2 and WebSocket
- Support redirects (from HTTP to HTTPS for example)

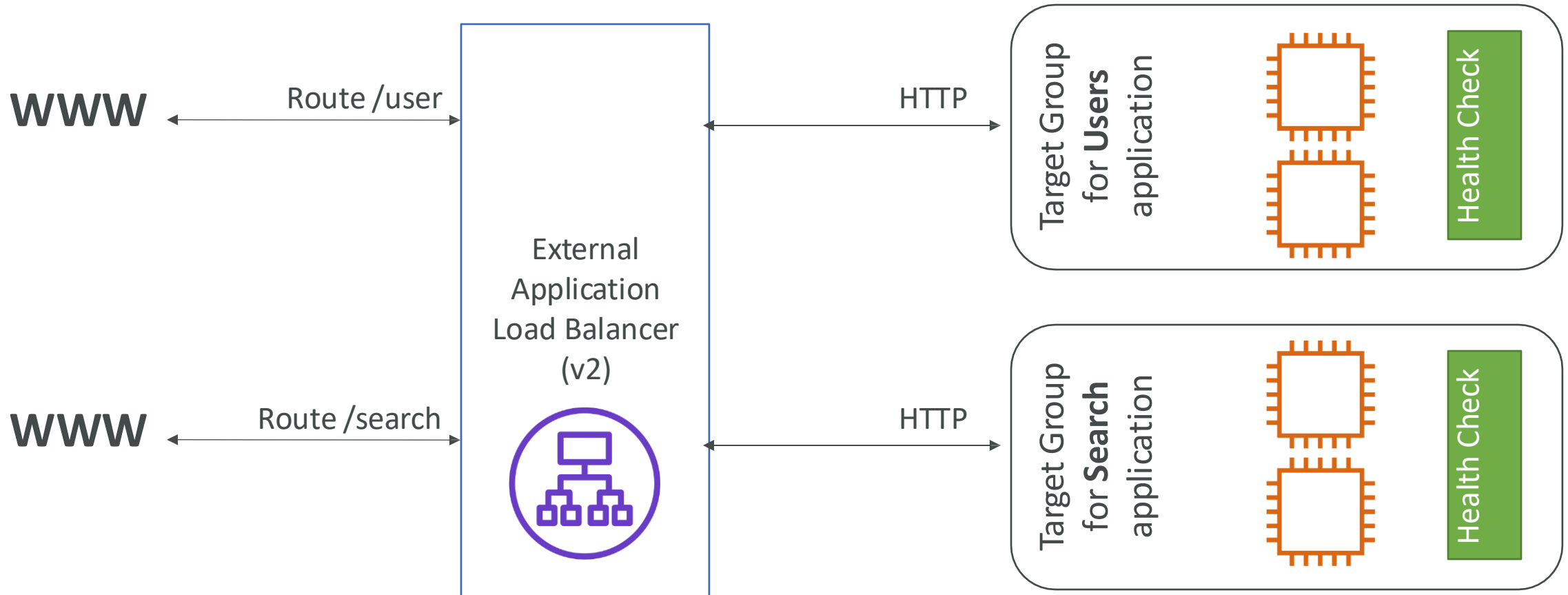
Application Load Balancer (v2)



- Routing tables to different target groups:
 - Routing based on path in URL (example.com/[users](#) & example.com/[orders](#))
 - Routing based on hostname in URL ([one.example.com](#) & [other.example.com](#))
 - Routing based on Query String, Headers (example.com/users?id=[123](#)&order=[false](#))
- ALB are a great fit for micro services & container-based application (example: Docker & Amazon ECS)
- Has a port mapping feature to redirect to a dynamic port in ECS
- In comparison, we'd need multiple Classic Load Balancer per application

Application Load Balancer (v2)

HTTP Based Traffic

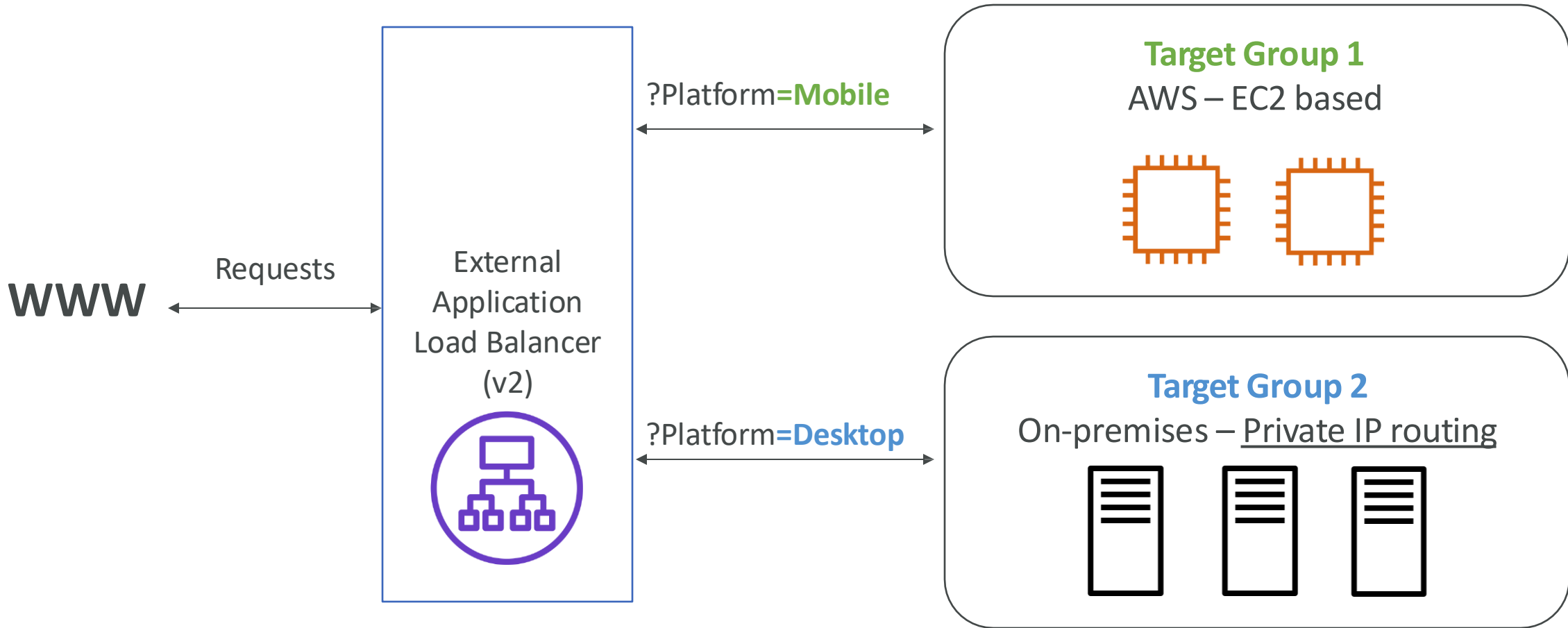


Application Load Balancer (v2)

Target Groups

- EC2 instances (can be managed by an Auto Scaling Group) - HTTP
 - ECS tasks (managed by ECS itself) - HTTP
 - Lambda functions - HTTP request is translated into a JSON event
 - IP Addresses - must be private IPs
-
- ALB can route to multiple target groups
 - Health checks are at the target group level

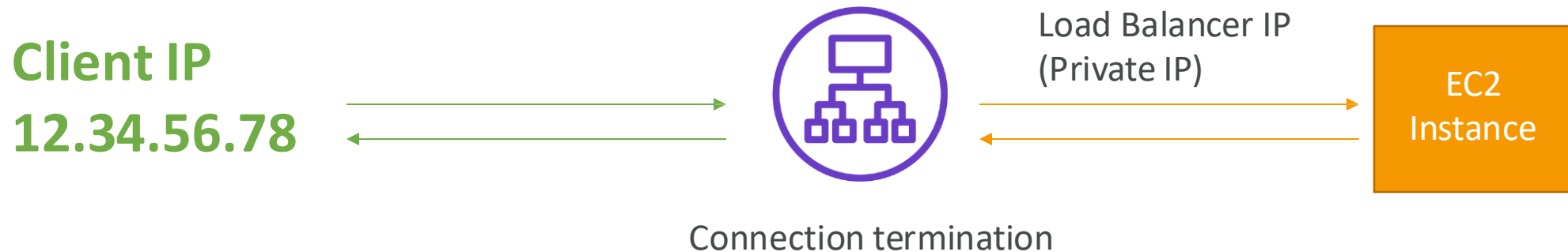
Application Load Balancer (v2) Query Strings/Parameters Routing

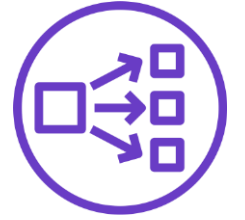


Application Load Balancer (v2)

Good to Know

- Fixed hostname (XXX.region.elb.amazonaws.com)
- The application servers don't see the IP of the client directly
 - The true IP of the client is inserted in the header X-Forwarded-For
 - We can also get Port (X-Forwarded-Port) and proto (X-Forwarded-Proto)



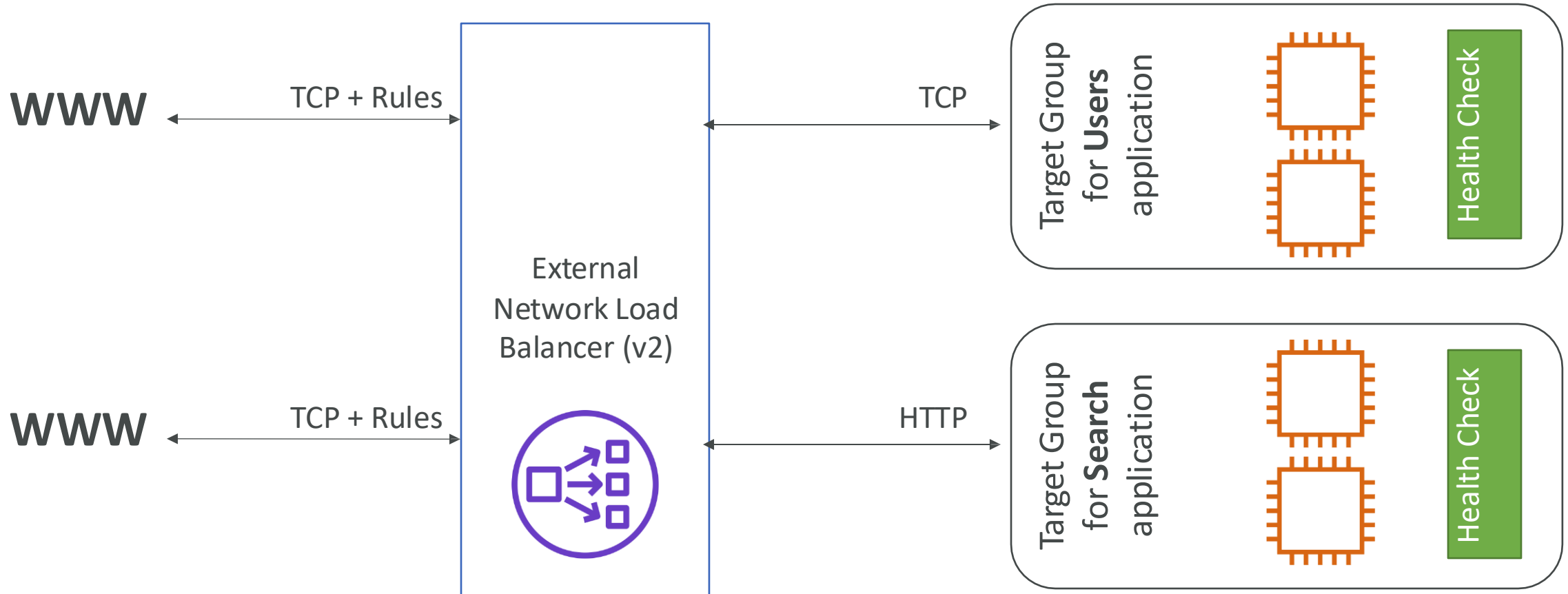


Network Load Balancer (v2)

- Network load balancers (Layer 4) allow to:
 - Forward TCP & UDP traffic to your instances
 - Handle millions of request per seconds
 - Ultra-low latency
- NLB has one static IP per AZ, and supports assigning Elastic IP (helpful for whitelisting specific IP)
- NLB are used for extreme performance, TCP or UDP traffic

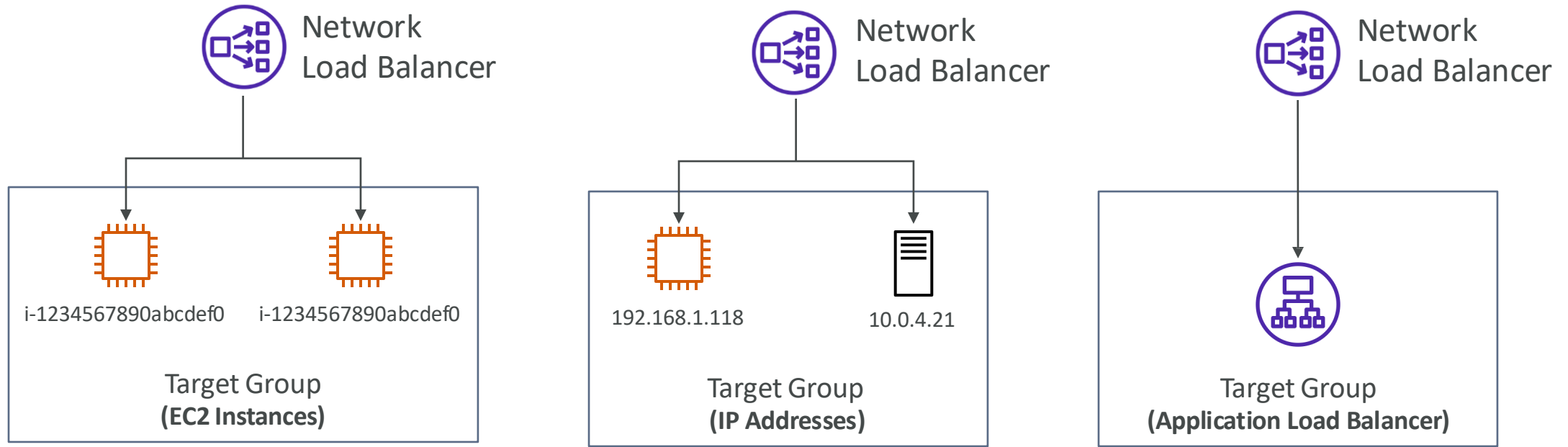
Network Load Balancer (v2)

TCP (Layer 4) Based Traffic



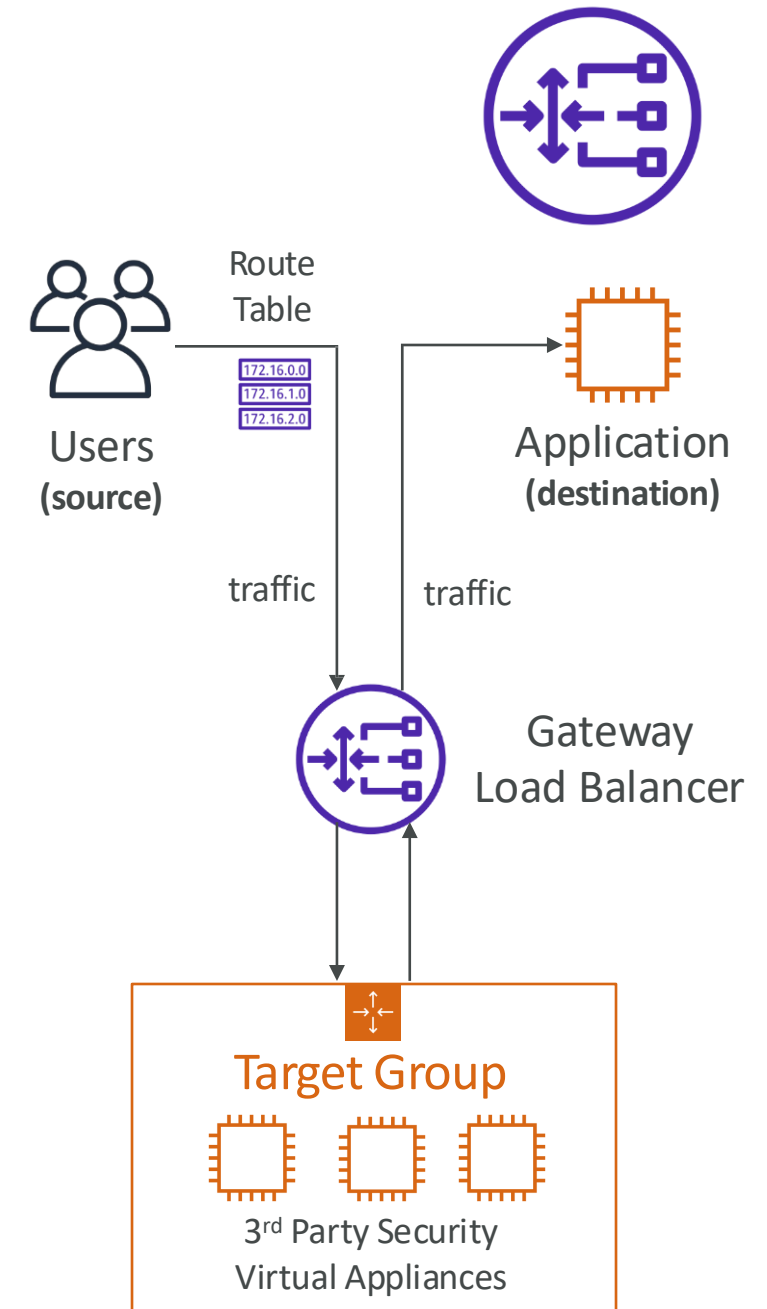
Network Load Balancer – Target Groups

- EC2 instances
- IP Addresses - must be private IPs
- Application Load Balancer
- Health Checks support the TCP, HTTP and HTTPS Protocols



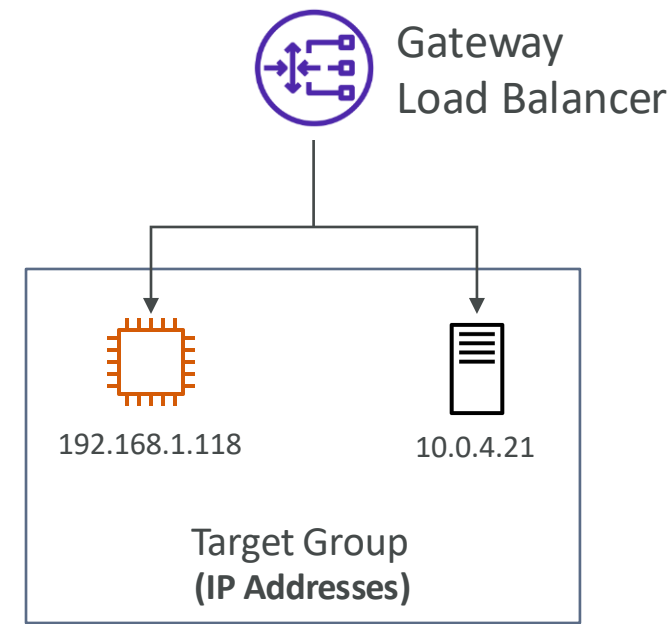
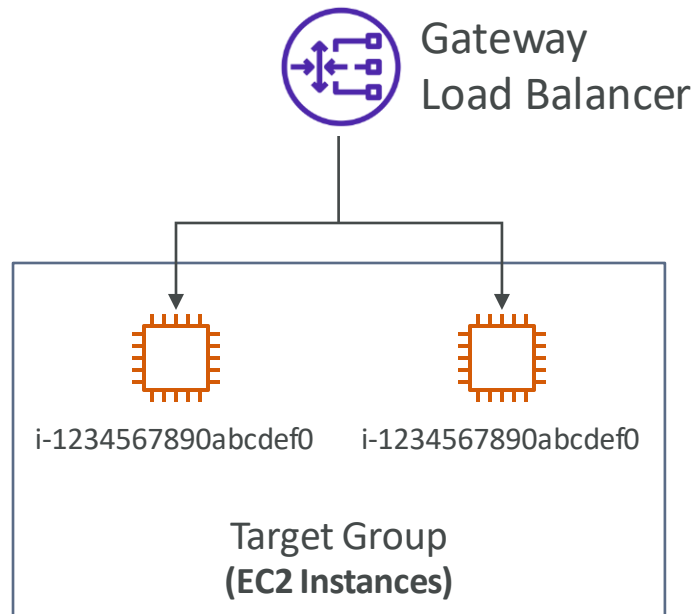
Gateway Load Balancer

- Deploy, scale, and manage a fleet of 3rd party network virtual appliances in AWS
- Example: Firewalls, Intrusion Detection and Prevention Systems, Deep Packet Inspection Systems, payload manipulation, ...
- Operates at Layer 3 (Network Layer) - IP Packets
- Combines the following functions:
 - Transparent Network Gateway - single entry/exit for all traffic
 - Load Balancer - distributes traffic to your virtual appliances
- Uses the GENEVE protocol on port 6081



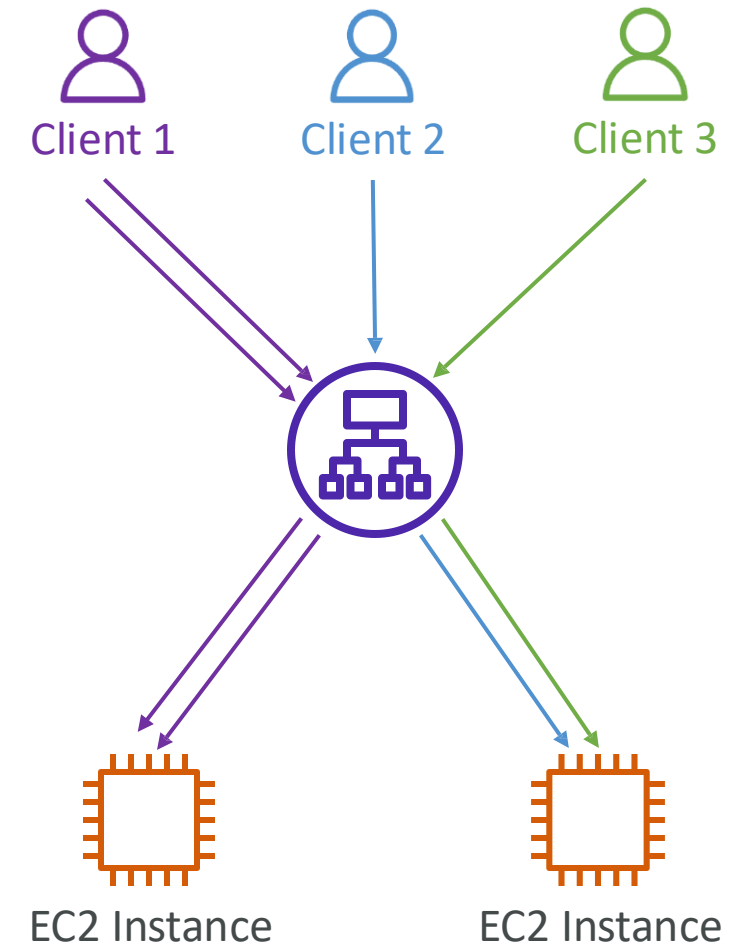
Gateway Load Balancer – Target Groups

- EC2 instances
- IP Addresses - must be private IPs



Sticky Sessions (Session Affinity)

- It is possible to implement stickiness so that the same client is always redirected to the same instance behind a load balancer
- This works for Classic Load Balancer, Application Load Balancer, and Network Load Balancer
- For both CLB & ALB, the “cookie” used for stickiness has an expiration date you control
- Use case: make sure the user doesn’t lose his session data
- Enabling stickiness may bring imbalance to the load over the backend EC2 instances



Sticky Sessions – Cookie Names

- Application-based Cookies
 - Custom cookie
 - Generated by the target
 - Can include any custom attributes required by the application
 - Cookie name must be specified individually for each target group
 - Don't use AWSALB, AWSALBAPP, or AWSALBTG (reserved for use by the ELB)
 - Application cookie
 - Generated by the load balancer
 - Cookie name is AWSALBAPP
- Duration-based Cookies
 - Cookie generated by the load balancer
 - Cookie name is AWSALB for ALB, AWSELB for CLB

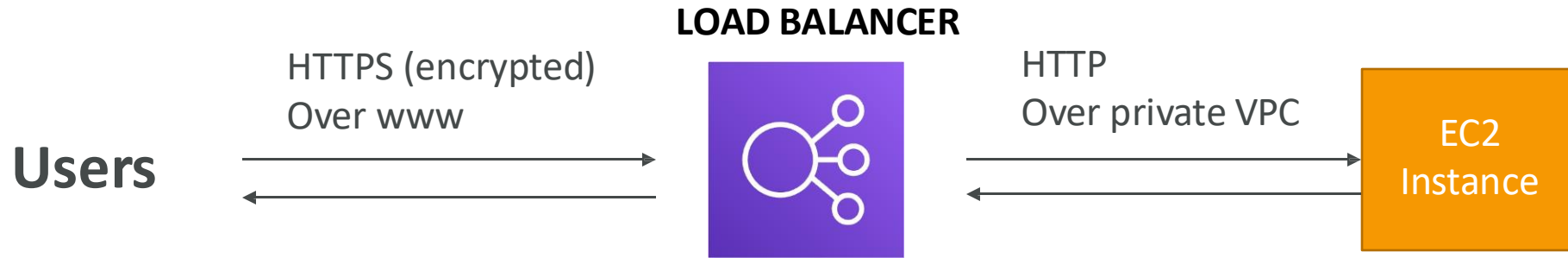
Cross-Zone Load Balancing

- Application Load Balancer
 - Enabled by default (can be disabled at the Target Group level)
 - No charges for inter AZ data
- Network Load Balancer & Gateway Load Balancer
 - Disabled by default
 - You pay charges (\$) for inter AZ data if enabled
- Classic Load Balancer
 - Disabled by default
 - No charges for inter AZ data if enabled

SSL/TLS - Basics

- An SSL Certificate allows traffic between your clients and your load balancer to be encrypted in transit (in-flight encryption)
- SSL refers to Secure Sockets Layer, used to encrypt connections
- TLS refers to Transport Layer Security, which is a newer version
- Nowadays, TLS certificates are mainly used, but people still refer as SSL
- Public SSL certificates are issued by Certificate Authorities (CA)
- Comodo, Symantec, GoDaddy, GlobalSign, Digicert, Letsencrypt, etc...
- SSL certificates have an expiration date (you set) and must be renewed

Load Balancer - SSL Certificates



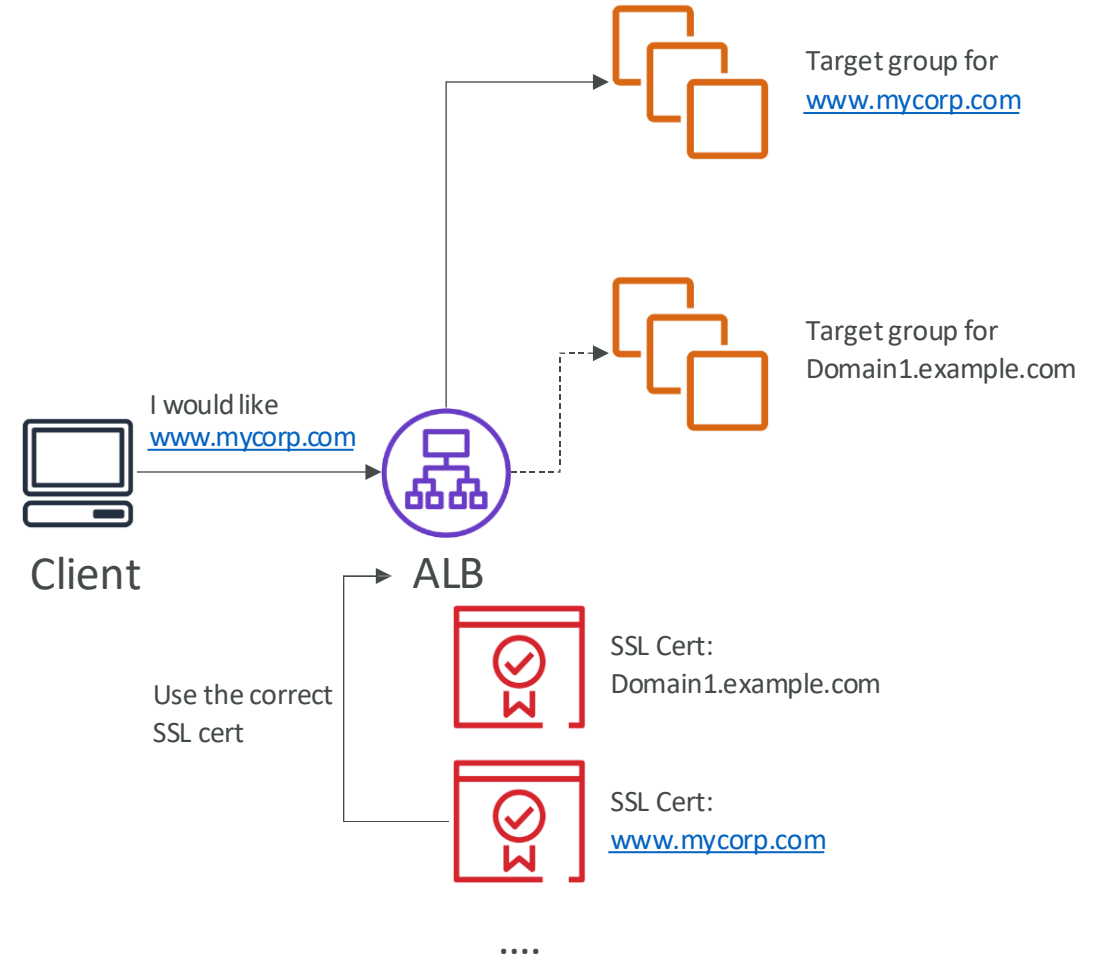
- The load balancer uses an X.509 certificate (SSL/TLS server certificate)
- You can manage certificates using ACM (AWS Certificate Manager)
- You can create upload your own certificates alternatively
- HTTPS listener:
 - You must specify a default certificate
 - You can add an optional list of certs to support multiple domains
 - Clients can use SNI (Server Name Indication) to specify the hostname they reach
 - Ability to specify a security policy to support older versions of SSL / TLS (legacy clients)

SSL – Server Name Indication (SNI)

- SNI solves the problem of loading multiple SSL certificates onto one web server (to serve multiple websites)
- It's a “newer” protocol, and requires the client to indicate the hostname of the target server in the initial SSL handshake
- The server will then find the correct certificate, or return the default one

Note:

- Only works for ALB & NLB (newer generation), CloudFront
- Does not work for CLB (older gen)

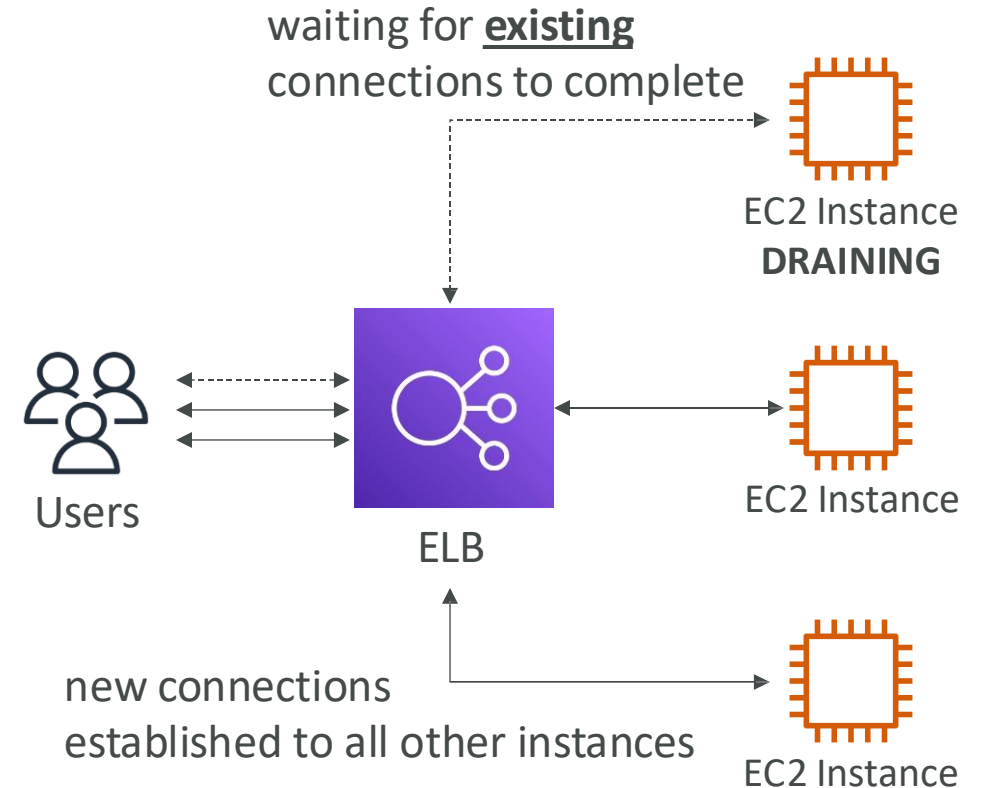


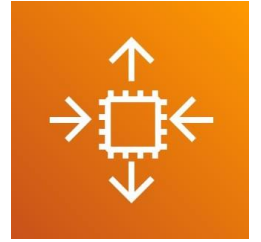
Elastic Load Balancers – SSL Certificates

- Classic Load Balancer (v1)
 - Support only one SSL certificate
 - Must use multiple CLB for multiple hostname with multiple SSL certificates
- Application Load Balancer (v2)
 - Supports multiple listeners with multiple SSL certificates
 - Uses Server Name Indication (SNI) to make it work
- Network Load Balancer (v2)
 - Supports multiple listeners with multiple SSL certificates
 - Uses Server Name Indication (SNI) to make it work

Connection Draining

- Feature naming
 - Connection Draining - for CLB
 - Deregistration Delay - for ALB & NLB
- Time to complete “in-flight requests” while the instance is de-registering or unhealthy
- Stops sending new requests to the EC2 instance which is de-registering
- Between 1 to 3600 seconds (default: 300 seconds)
- Can be disabled (set value to 0)
- Set to a low value if your requests are short

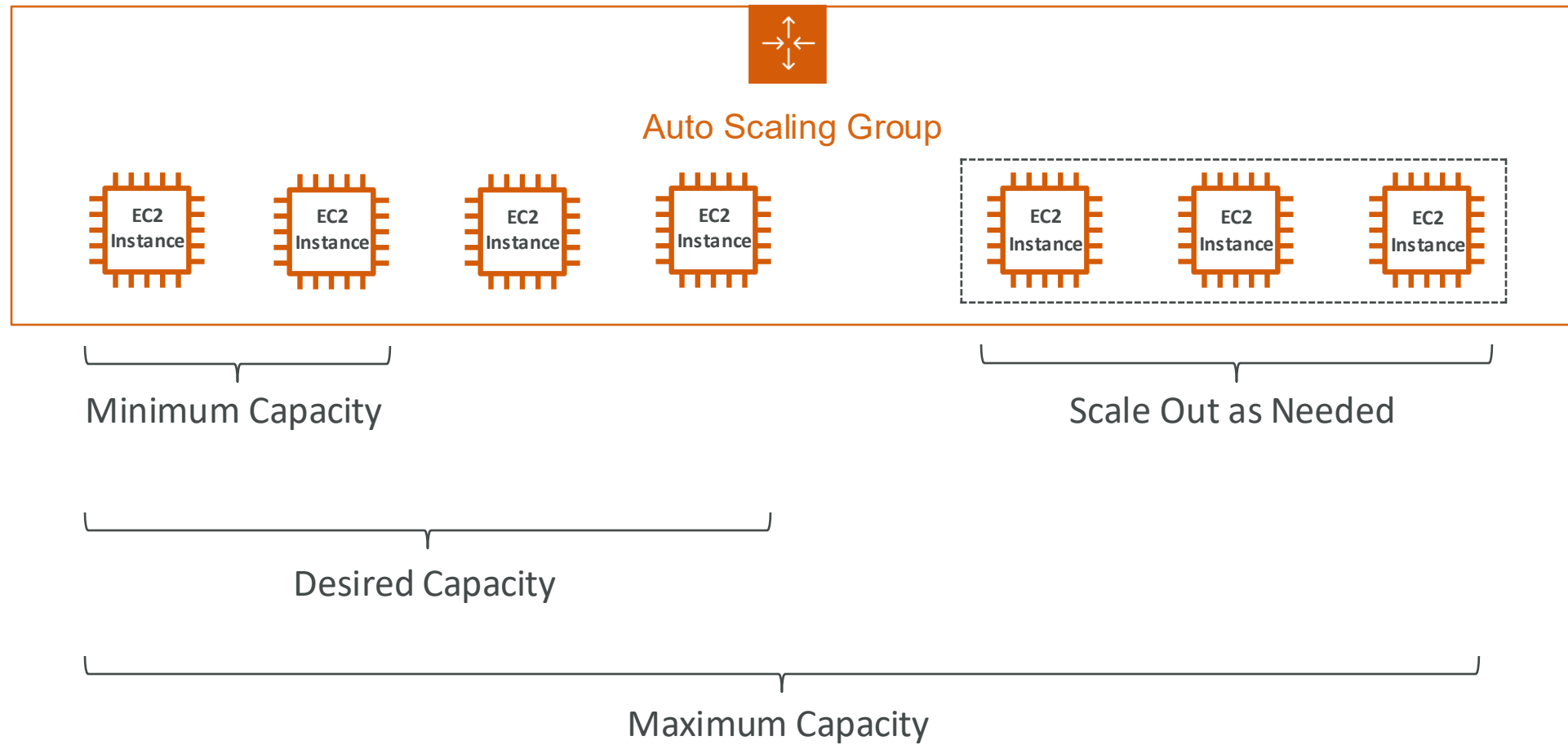




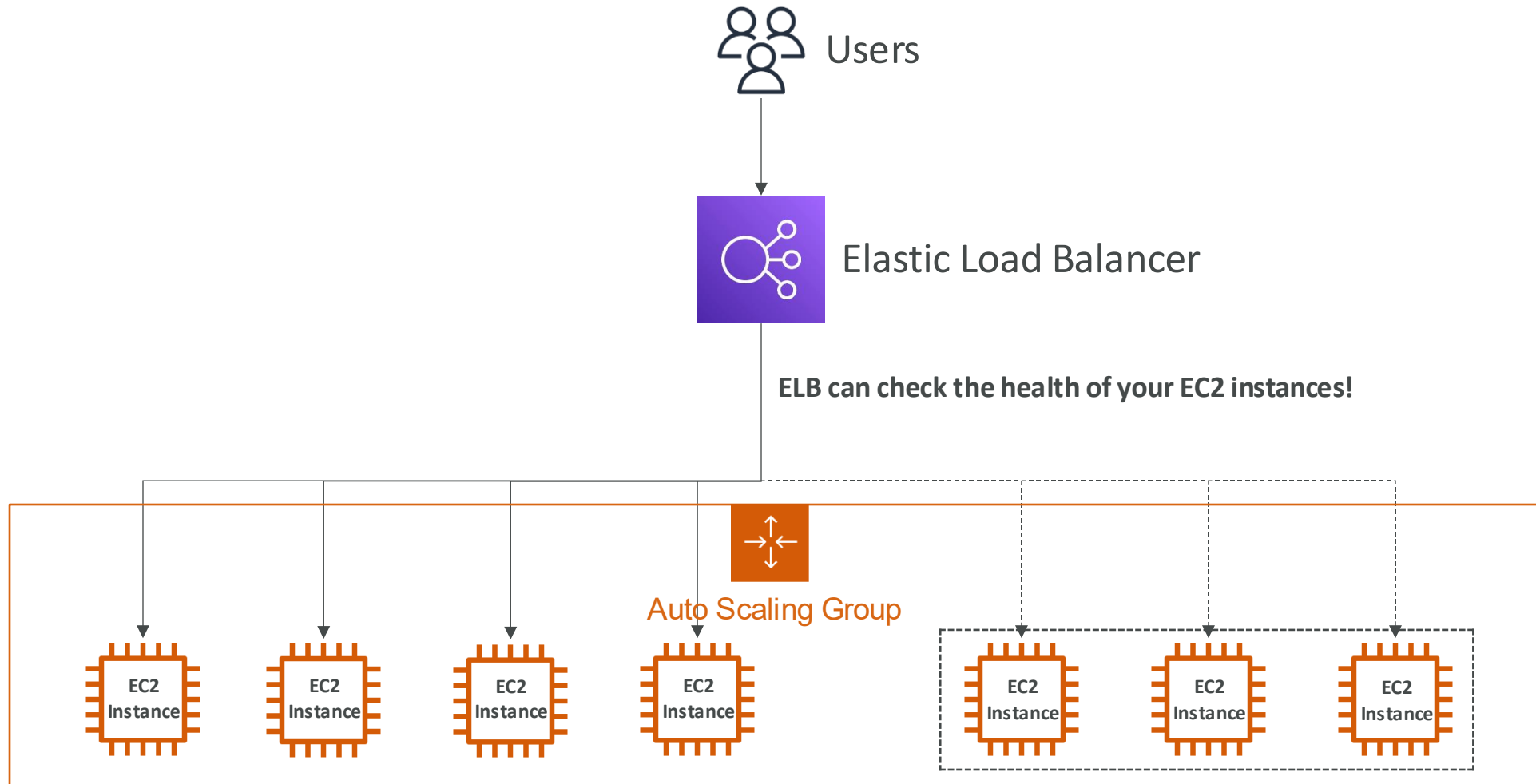
What's an Auto Scaling Group?

- In real-life, the load on your websites and application can change
- In the cloud, you can create and get rid of servers very quickly
- The goal of an Auto Scaling Group (ASG) is to:
 - Scale out (add EC2 instances) to match an increased load
 - Scale in (remove EC2 instances) to match a decreased load
 - Ensure we have a minimum and a maximum number of EC2 instances running
 - Automatically register new instances to a load balancer
 - Re-create an EC2 instance in case a previous one is terminated (ex: if unhealthy)
- ASG are free (you only pay for the underlying EC2 instances)

Auto Scaling Group in AWS

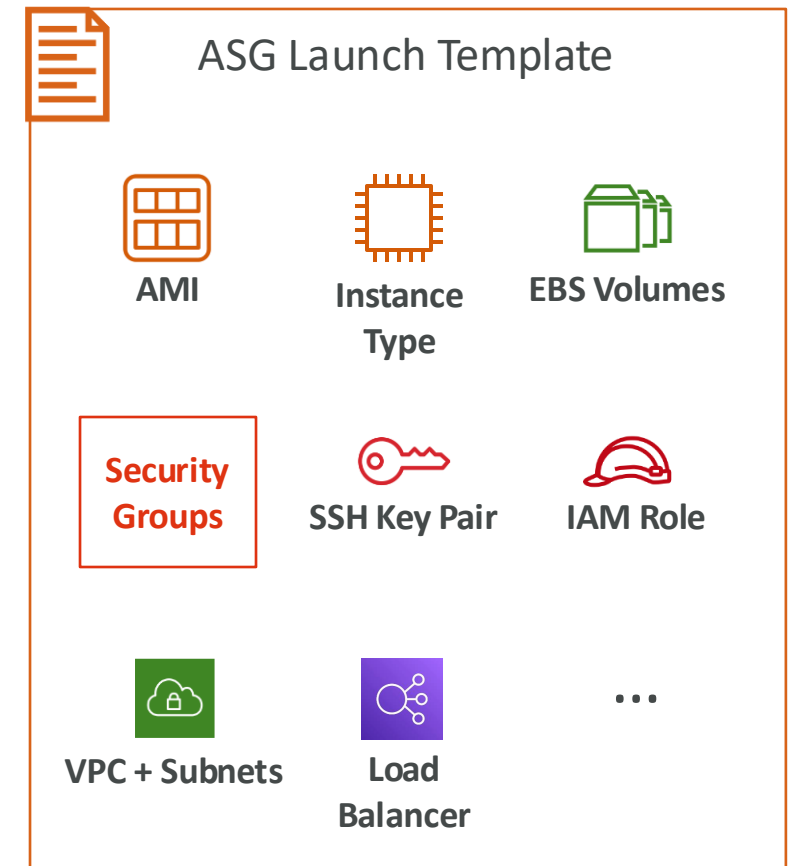


Auto Scaling Group in AWS With Load Balancer



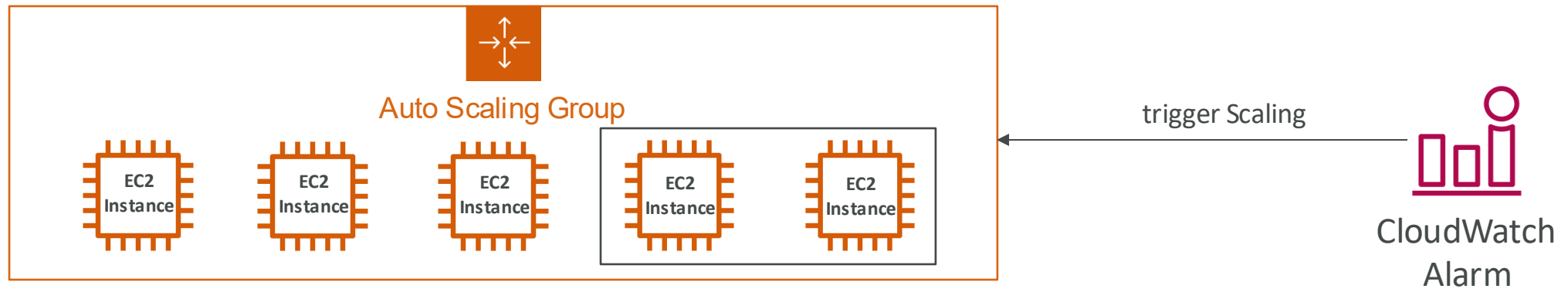
Auto Scaling Group Attributes

- A Launch Template (older “Launch Configurations” are deprecated)
 - AMI + Instance Type
 - EC2 User Data
 - EBS Volumes
 - Security Groups
 - SSH Key Pair
 - IAM Roles for your EC2 Instances
 - Network + Subnets Information
 - Load Balancer Information
- Min Size / Max Size / Initial Capacity
- Scaling Policies



Auto Scaling - CloudWatch Alarms & Scaling

- It is possible to scale an ASG based on CloudWatch alarms
- An alarm monitors a metric (such as Average CPU, or a custom metric)
- Metrics such as Average CPU are computed for the overall ASG instances
- Based on the alarm:
 - We can create scale-out policies (increase the number of instances)
 - We can create scale-in policies (decrease the number of instances)

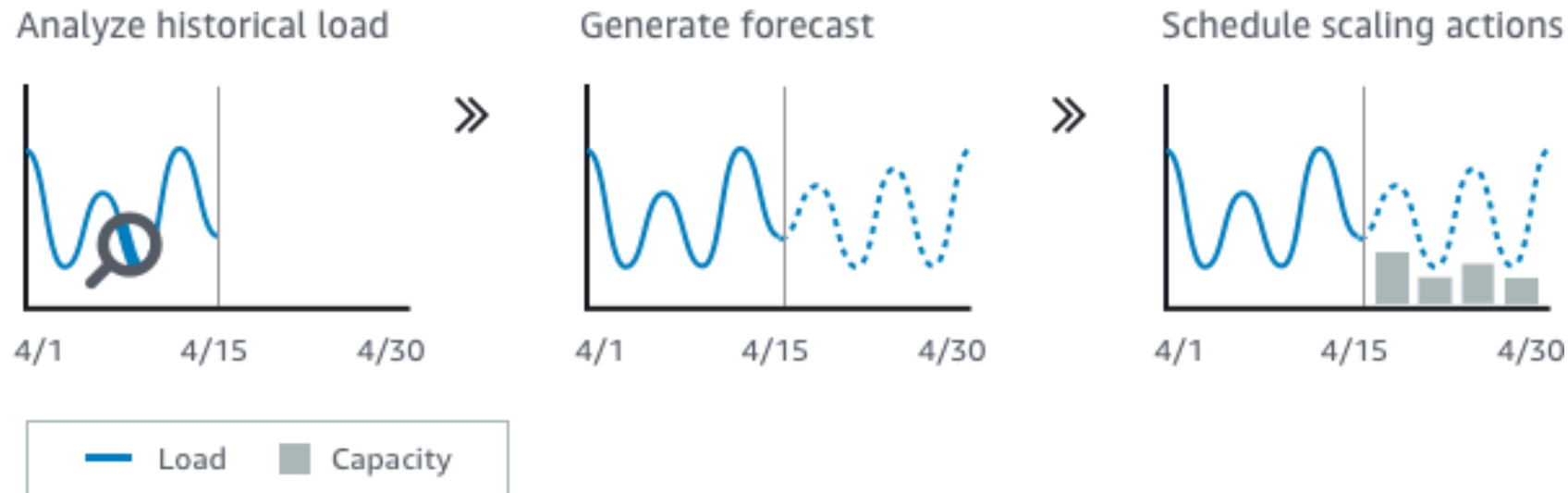


Auto Scaling Groups – Scaling Policies

- Dynamic Scaling
 - Target Tracking Scaling
 - Simple to set-up
 - Example: I want the average ASG CPU to stay at around 40%
 - Simple / Step Scaling
 - When a CloudWatch alarm is triggered (example CPU > 70%), then add 2 units
 - When a CloudWatch alarm is triggered (example CPU < 30%), then remove 1
- Scheduled Scaling
 - Anticipate a scaling based on known usage patterns
 - Example: increase the min capacity to 10 at 5 pm on Fridays

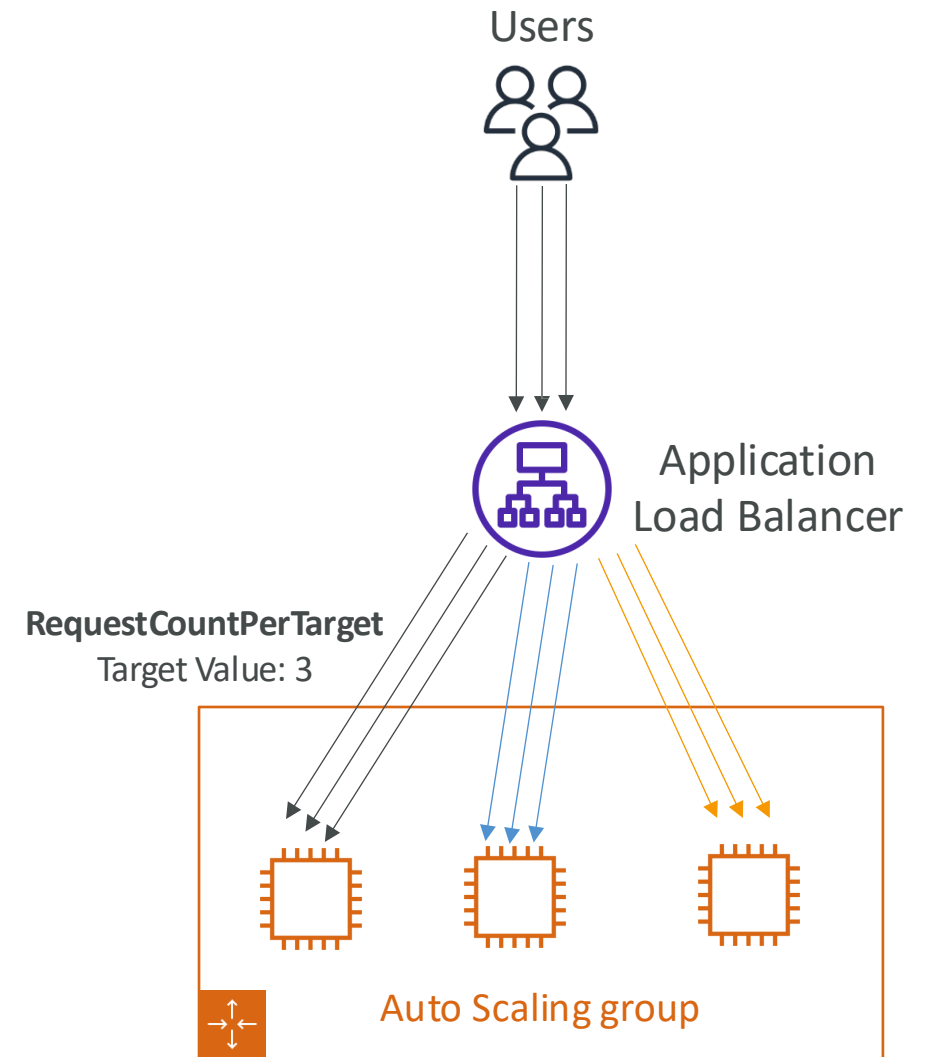
Auto Scaling Groups – Scaling Policies

- Predictive scaling: continuously forecast load and schedule scaling ahead



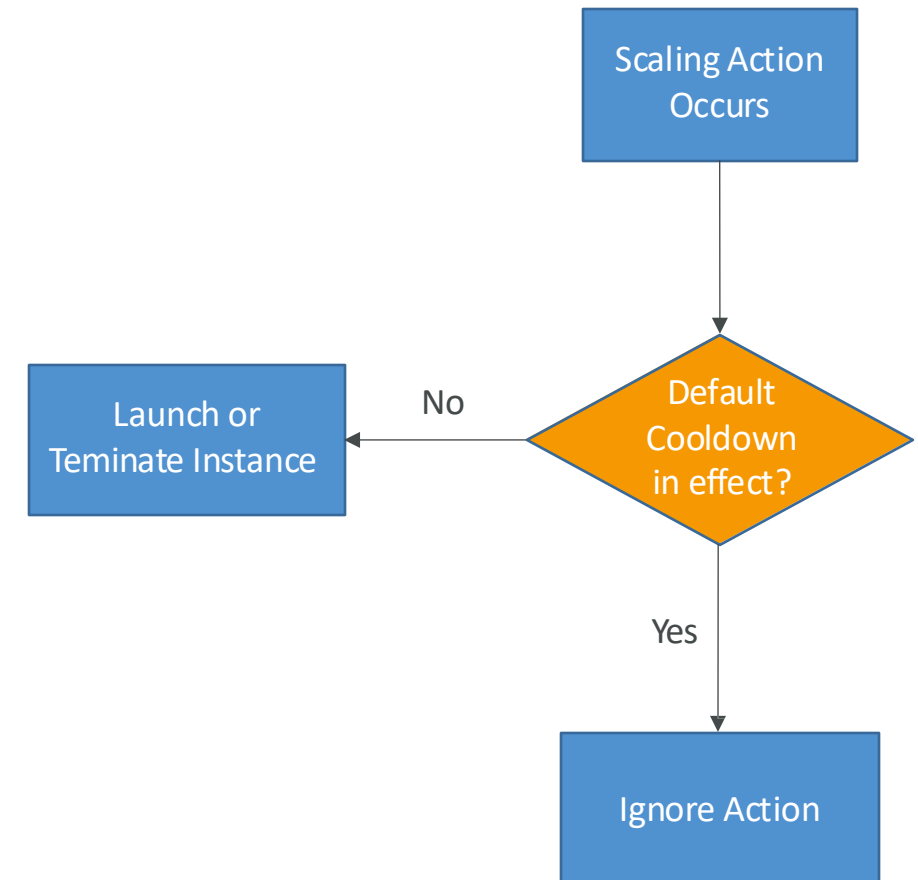
Good metrics to scale on

- CPUUtilization: Average CPU utilization across your instances
- RequestCountPerTarget: to make sure the number of requests per EC2 instances is stable
- Average Network In / Out (if you're application is network bound)
- Any custom metric (that you push using CloudWatch)



Auto Scaling Groups - Scaling Cooldowns

- After a scaling activity happens, you are in the cooldown period (default 300 seconds)
- During the cooldown period, the ASG will not launch or terminate additional instances (to allow for metrics to stabilize)
- Advice: Use a ready-to-use AML to reduce configuration time in order to be serving request faster and reduce the cooldown period



Amazon EC2 – Instance Storage



What's an EBS Volume?

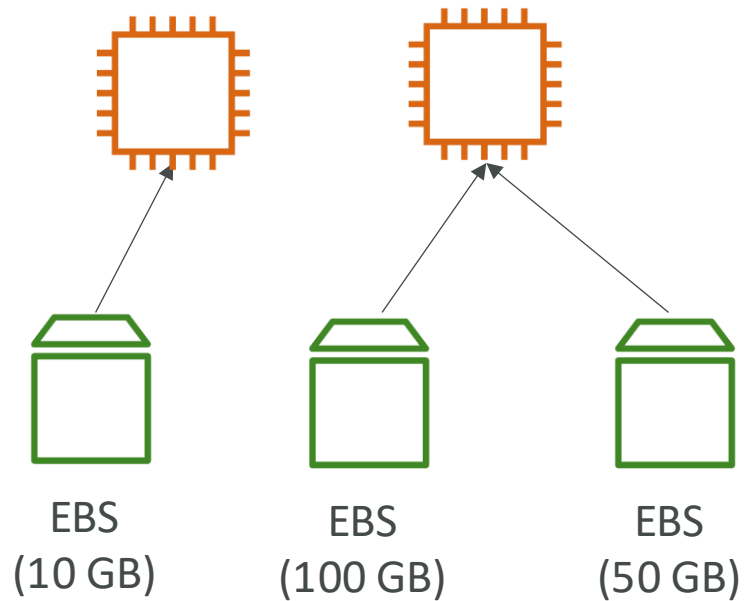
- An **EBS (Elastic Block Store) Volume** is a **network** drive you can attach to your instances while they run
- It allows your instances to persist data, even after their termination
- They can only be mounted to one instance at a time (at the CCP level)
- They are bound to a specific availability zone
- Analogy: Think of them as a “network USB stick”

EBSVolume

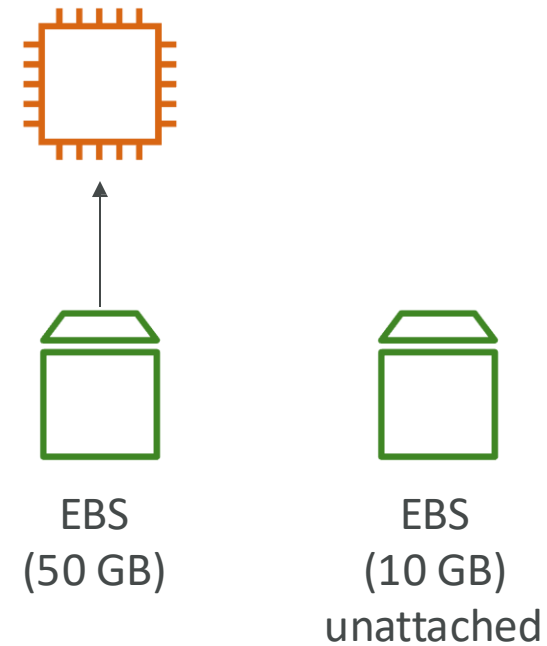
- It's a network drive (i.e. not a physical drive)
 - It uses the network to communicate the instance, which means there might be a bit of latency
 - It can be detached from an EC2 instance and attached to another one quickly
- It's locked to an Availability Zone (AZ)
 - An EBSVolume in us-east-1a cannot be attached to us-east-1b
 - To move a volume across, you first need to snapshot it
- Have a provisioned capacity (size in GBs, and IOPS)
 - You get billed for all the provisioned capacity
 - You can increase the capacity of the drive over time

EBS Volume - Example

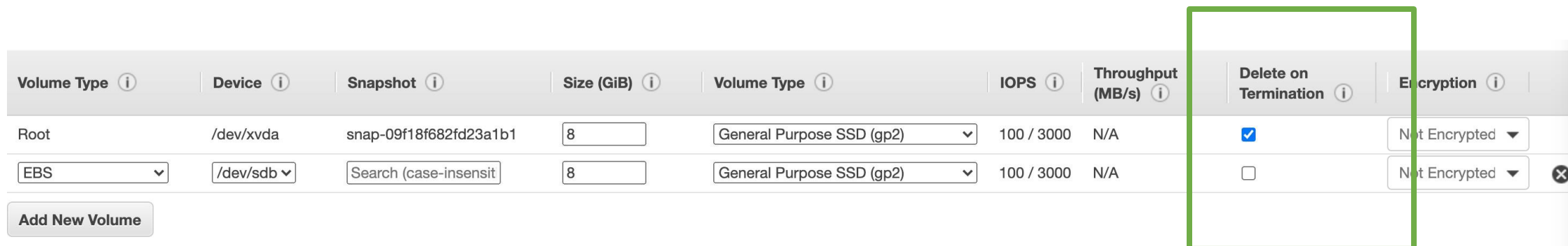
US-EAST-1A



US-EAST-1B



EBS – Delete on Termination attribute



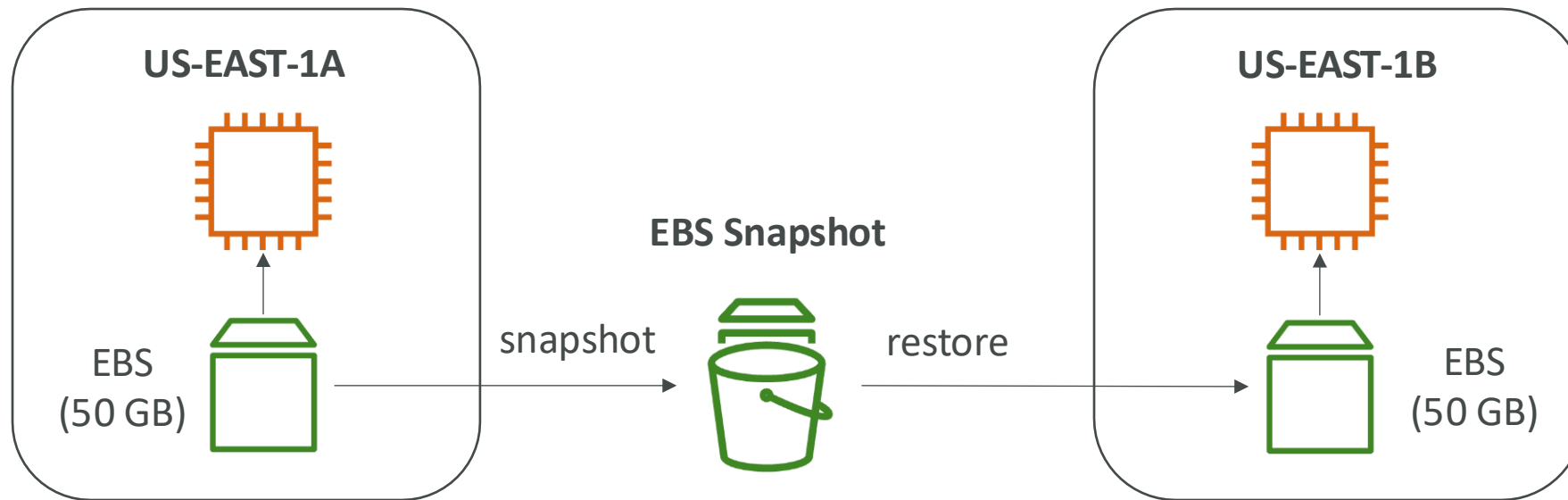
Volume Type ⓘ	Device ⓘ	Snapshot ⓘ	Size (GiB) ⓘ	Volume Type ⓘ	IOPS ⓘ	Throughput (MB/s) ⓘ	Delete on Termination ⓘ	Encryption ⓘ
Root	/dev/xvda	snap-09f18f682fd23a1b1	8	General Purpose SSD (gp2) ▾	100 / 3000	N/A	☒	Not Encrypted ▾
EBS ▾	/dev/sdb ▾	Search (case-insensit	8	General Purpose SSD (gp2) ▾	100 / 3000	N/A	☐	Not Encrypted ▾

Add New Volume

- Controls the EBS behaviour when an EC2 instance terminates
 - By default, the root EBS volume is deleted (attribute enabled)
 - By default, any other attached EBS volume is not deleted (attribute disabled)
- This can be controlled by the AWS console / AWS CLI
- Use case: preserve root volume when instance is terminated

EBS Snapshots

- Make a backup (snapshot) of your EBS volume at a point in time
- Not necessary to detach volume to do snapshot, but recommended
- Can copy snapshots across AZ or Region



EBS Snapshots Features

- EBS Snapshot Archive

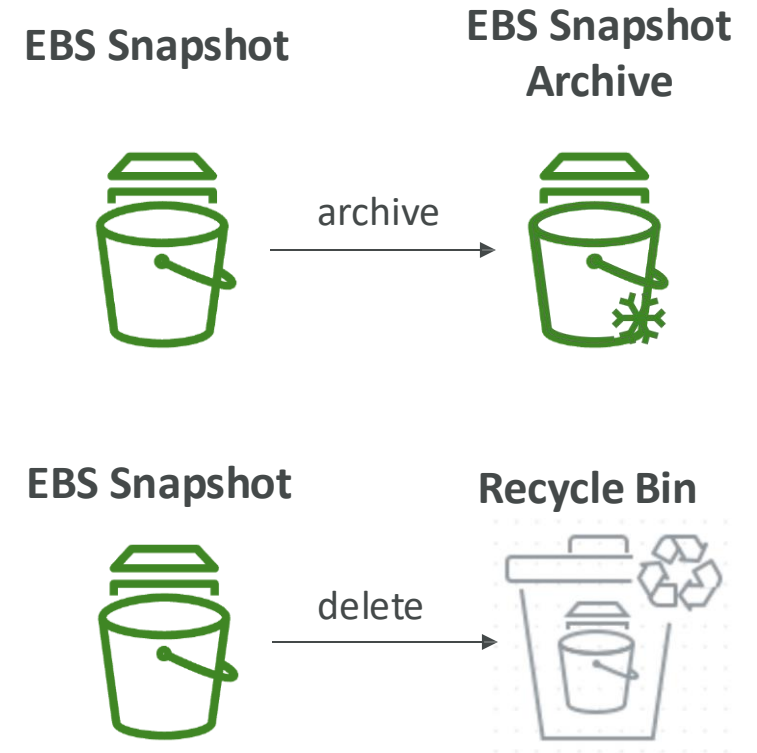
- Move a Snapshot to an "archive tier" that is 75% cheaper
- Takes within 24 to 72 hours for restoring the archive

- Recycle Bin for EBS Snapshots

- Setup rules to retain deleted snapshots so you can recover them after an accidental deletion
- Specify retention (from 1 day to 1 year)

- Fast Snapshot Restore (FSR)

- Force full initialization of snapshot to have no latency on the first use (\$\$\$)



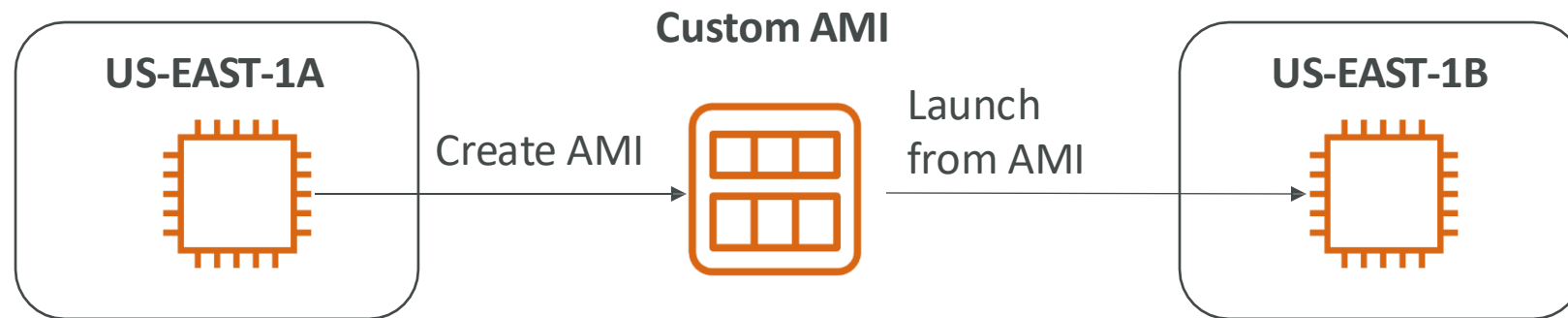


AMI Overview

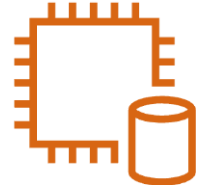
- AMI = Amazon Machine Image
- AMI are a customization of an EC2 instance
 - You add your own software, configuration, operating system, monitoring...
 - Faster boot / configuration time because all your software is pre-packaged
- AMI are built for a specific region (and can be copied across regions)
- You can launch EC2 instances from:
 - A Public AMI: AWS provided
 - Your own AMI: you make and maintain them yourself
 - An AWS Marketplace AMI: an AMI someone else made (and potentially sells)

AMI Process (from an EC2 instance)

- Start an EC2 instance and customize it
- Stop the instance (for data integrity)
- Build an AMI - this will also create EBS snapshots
- Launch instances from other AMIs



EC2 Instance Store



- EBS volumes are network drives with good but “limited” performance
- If you need a high-performance hardware disk, use EC2 Instance Store
- Better I/O performance
- EC2 Instance Store lose their storage if they're stopped (ephemeral)
- Good for buffer / cache / scratch data / temporary content
- Risk of data loss if hardware fails
- Backups and Replication are your responsibility

Local EC2 Instance Store

Very high IOPS

Instance Size	100% Random Read IOPS	Write IOPS
i3.large *	100,125	35,000
i3.xlarge *	206,250	70,000
i3.2xlarge	412,500	180,000
i3.4xlarge	825,000	360,000
i3.8xlarge	1.65 million	720,000
i3.16xlarge	3.3 million	1.4 million
i3.metal	3.3 million	1.4 million
i3en.large *	42,500	32,500
i3en.xlarge *	85,000	65,000
i3en.2xlarge *	170,000	130,000
i3en.3xlarge	250,000	200,000
i3en.6xlarge	500,000	400,000
i3en.12xlarge	1 million	800,000
i3en.24xlarge	2 million	1.6 million
i3en.metal	2 million	1.6 million

EBS Volume Types

- EBS Volumes come in 6 types
 - **gp2 / gp3 (SSD)**: General purpose SSD volume that balances price and performance for a wide variety of workloads
 - **io1 / io2 Block Express (SSD)**: Highest-performance SSD volume for mission-critical low-latency or high-throughput workloads
 - **st1 (HDD)**: Low cost HDD volume designed for frequently accessed, throughput-intensive workloads
 - **sc1 (HDD)**: Lowest cost HDD volume designed for less frequently accessed workloads
- EBS Volumes are characterized in Size | Throughput | IOPS (I/O Ops Per Sec)
- When in doubt always consult the AWS documentation - it's good!
- Only gp2/gp3 and io1/io2 Block Express can be used as boot volumes

EBS Volume Types Use cases

General Purpose SSD

- Cost effective storage, low-latency
- System boot volumes, Virtual desktops, Development and test environments
- 1 GiB - 16 TiB
- gp3:
 - Baseline of 3,000 IOPS and throughput of 125 MiB/s
 - Can increase IOPS up to 16,000 and throughput up to 1000 MiB/s independently
- gp2:
 - Small gp2 volumes can burst IOPS to 3,000
 - Size of the volume and IOPS are linked, max IOPS is 16,000
 - 3 IOPS per GiB, means at 5,334 GiB we are at the max IOPS

EBS Volume Types Use cases

Provisioned IOPS (PIOPS) SSD

- Critical business applications with sustained IOPS performance
- Or applications that need more than 16,000 IOPS
- Great for databases workloads (sensitive to storage perf and consistency)
- io1 (4 GiB - 16 TiB):
 - Max PIOPS: 64,000 for Nitro EC2 instances & 32,000 for other
 - Can increase PIOPS independently from storage size
- io2 Block Express (4 GiB - 64 TiB):
 - Sub-millisecond latency
 - Max PIOPS: 256,000 with an IOPS:GiB ratio of 1,000:1
- Supports EBS Multi-attach

EBS Volume Types Use cases

Hard Disk Drives (HDD)

- Cannot be a boot volume
- 125 GiB to 16 TiB
- Throughput Optimized HDD (st1)
 - Big Data, Data Warehouses, Log Processing
 - Max throughput 500 MiB/s - max IOPS 500
- Cold HDD (sc1):
 - For data that is infrequently accessed
 - Scenarios where lowest cost is important
 - Max throughput 250 MiB/s - max IOPS 250

EBS – Volume Types Summary

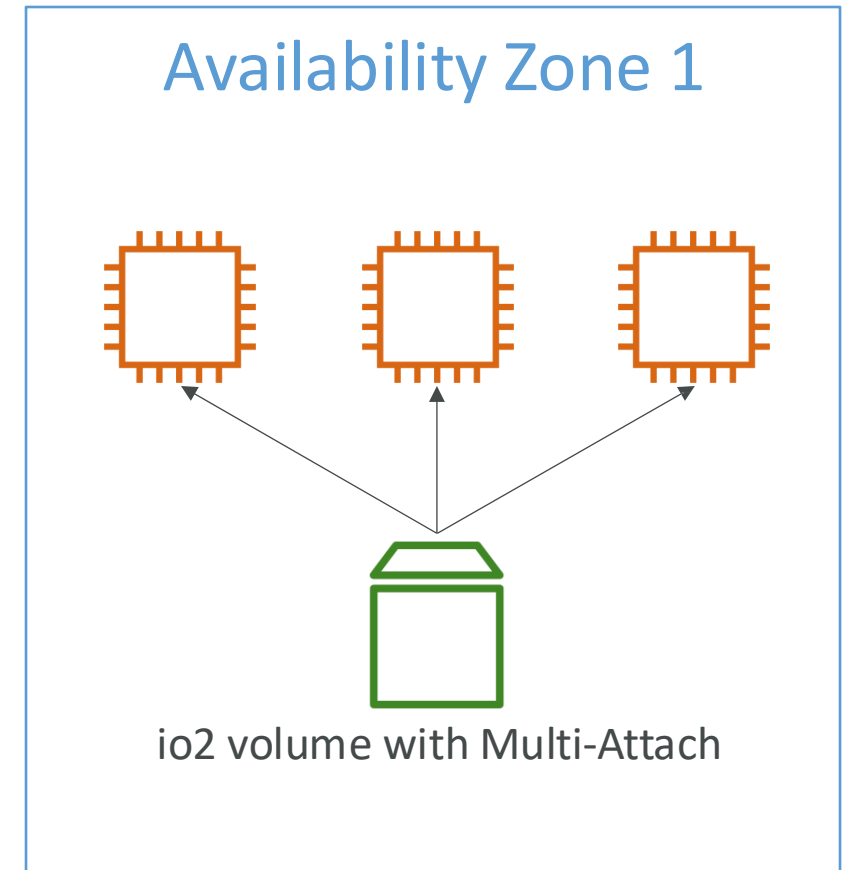
	General Purpose SSD volumes		Provisioned IOPS SSD volumes	
Volume type	gp3	gp2	io2 Block Express ³	io1
Durability	99.8% - 99.9% durability (0.1% - 0.2% annual failure rate)		99.999% durability (0.001% annual failure rate)	99.8% - 99.9% durability (0.1% - 0.2% annual failure rate)
Use cases	- Transactional workloads - Virtual desktops - Medium-sized, single-instance databases - Low-latency interactive applications - Boot volumes - Development and test environments		Workloads that require: - Sub-millisecond latency - Sustained IOPS performance - More than 64,000 IOPS or 1,000 MiB/s of throughput	- Workloads that require sustained IOPS performance or more than 16,000 IOPS - I/O-intensive database workloads
Volume size	1 GiB - 16 TiB		4 GiB - 64 TiB ⁴	4 GiB - 16 TiB
Max IOPS per volume (16 KiB I/O)	16,000		256,000 ⁵	64,000
Max throughput per volume	1,000 MiB/s	250 MiB/s ¹	4,000 MiB/s	1,000 MiB/s ²
Amazon EBS Multi-attach	Not supported		Supported	
NVMe reservations	Not supported		Supported	Not supported
Boot volume	Supported			

	Throughput Optimized HDD volumes	Cold HDD volumes
Volume type	st1	sc1
Durability	99.8% - 99.9% durability (0.1% - 0.2% annual failure rate)	
Use cases	- Big data - Data warehouses - Log processing	- Throughput-oriented storage for data that is infrequently accessed - Scenarios where the lowest storage cost is important
Volume size	125 GiB - 16 TiB	
Max IOPS per volume (1 MiB I/O)	500	250
Max throughput per volume	500 MiB/s	250 MiB/s
Amazon EBS Multi-attach	Not supported	
Boot volume	Not supported	

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html#solid-state-drives>

EBS Multi-Attach – io1/io2 family

- Attach the same EBS volume to multiple EC2 instances in the same AZ
- Each instance has full read & write permissions to the high-performance volume
- Use case:
 - Achieve higher application availability in clustered Linux applications (ex: Teradata)
 - Applications must manage concurrent write operations
- Up to 16 EC2 Instances at a time
- Must use a file system that's cluster-aware (not XFS, EXT4, etc...)



EBS Encryption

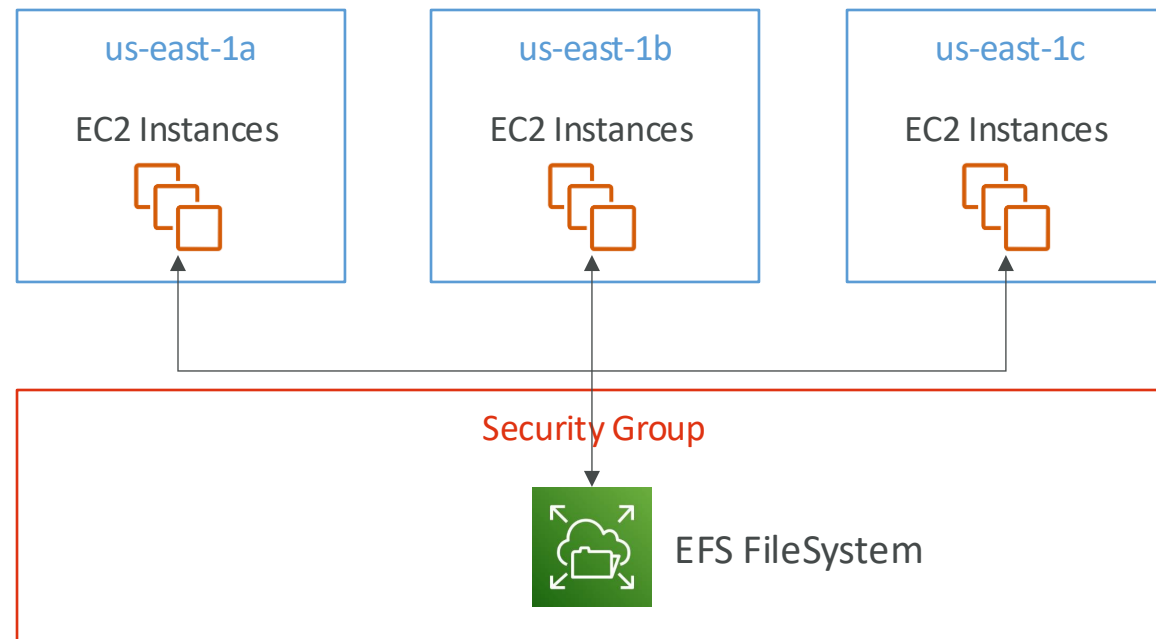
- When you create an encrypted EBS volume, you get the following:
 - Data at rest is encrypted inside the volume
 - All the data in flight moving between the instance and the volume is encrypted
 - All snapshots are encrypted
 - All volumes created from the snapshot
- Encryption and decryption are handled transparently (you have nothing to do)
- Encryption has a minimal impact on latency
- EBS Encryption leverages keys from KMS (AES-256)
- Copying an unencrypted snapshot allows encryption
- Snapshots of encrypted volumes are encrypted

Encryption: encrypt an unencrypted EBS volume

- Create an EBS snapshot of the volume
- Encrypt the EBS snapshot (using copy)
- Create new ebs volume from the snapshot (the volume will also be encrypted)
- Now you can attach the encrypted volume to the original instance

Amazon EFS – Elastic File System

- Managed NFS (network file system) that can be mounted on many EC2
- EFS works with EC2 instances in multi-AZ
- Highly available, scalable, expensive (3x gp2), pay per use



Amazon EFS— Elastic File System

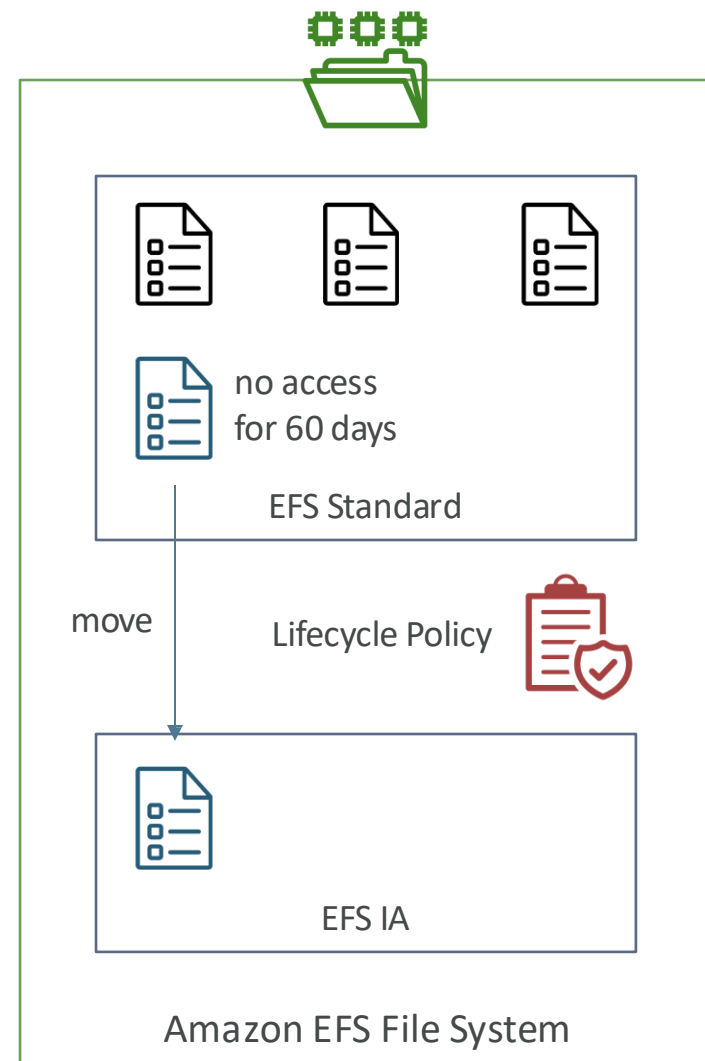
- Use cases: content management, web serving, data sharing, Wordpress
 - Uses NFSv4.1 protocol
 - Uses security group to control access to EFS
 - Compatible with Linux based AMI (not Windows)
 - Encryption at rest using KMS
-
- POSIX file system (~Linux) that has a standard file API
 - File system scales automatically, pay-per-use, no capacity planning!

EFS— Performance & Storage Classes

- EFS Scale
 - 1000s of concurrent NFS clients, 10 GB+ /s throughput
 - Grow to Petabyte-scale network file system, automatically
- Performance Mode (set at EFS creation time)
 - General Purpose (default) - latency-sensitive use cases (web server, CMS, etc...)
 - Max I/O - higher latency, throughput, highly parallel (big data, media processing)
- Throughput Mode
 - Bursting - 1 TB = 50MiB/s + burst of up to 100MiB/s
 - Provisioned - set your throughput regardless of storage size, ex: 1 GiB/s for 1 TB storage
 - Elastic - automatically scales throughput up or down based on your workloads
 - Up to 3GiB/s for reads and 1GiB/s for writes
 - Used for unpredictable workloads

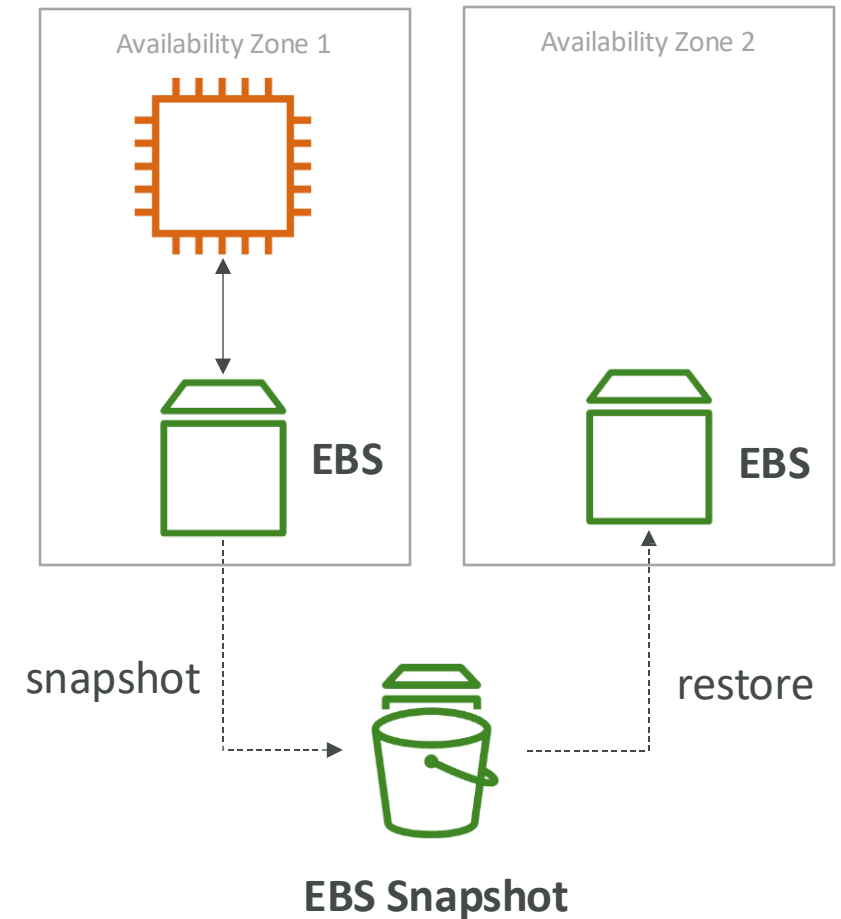
EFS – Storage Classes

- Storage Tiers (lifecycle management feature - move file after N days)
 - Standard: for frequently accessed files
 - Infrequent access (EFS-IA): cost to retrieve files, lower price to store.
 - Archive: rarely accessed data (few times each year), 50% cheaper
 - Implement lifecycle policies to move files between storage tiers
- Availability and durability
 - Standard: Multi-AZ, great for prod
 - One Zone: One AZ, great for dev, backup enabled by default, compatible with IA (EFS One Zone-IA)
- Over 90% in cost savings



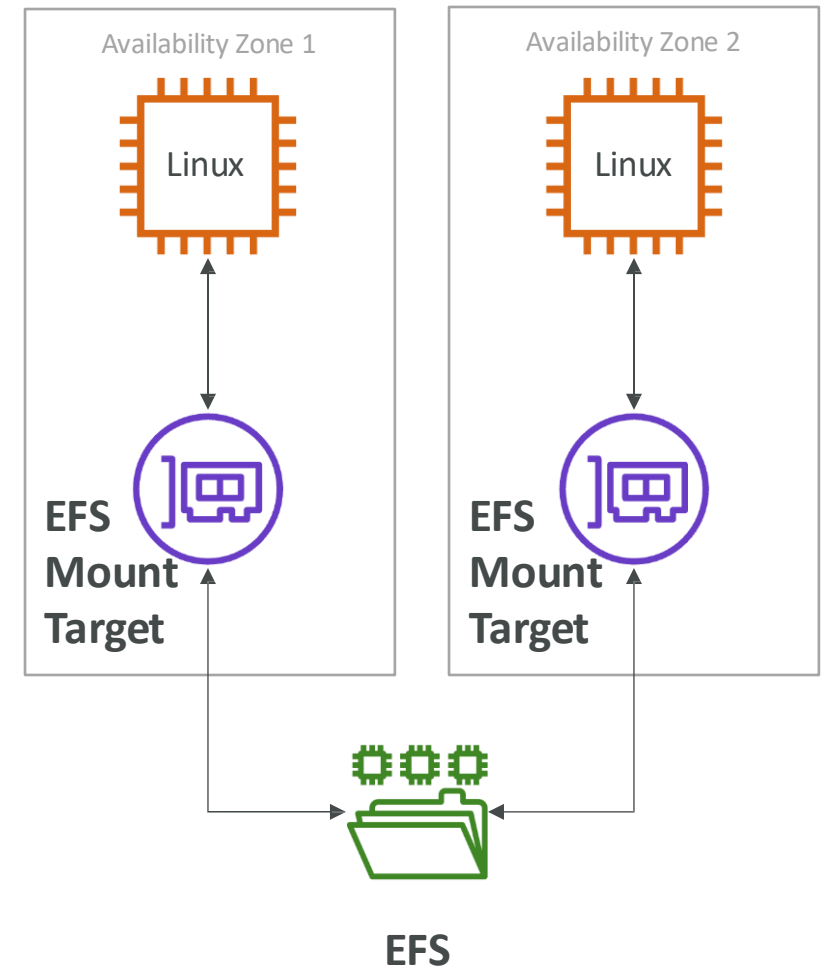
EBS vs EFS — Elastic Block Storage

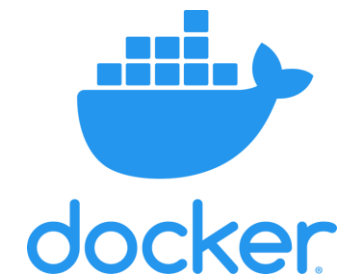
- EBS volumes...
 - one instance (except multi-attach io1/io2)
 - are locked at the Availability Zone (AZ) level
 - gp2: IO increases if the disk size increases
 - gp3 & io1: can increase IO independently
- To migrate an EBS volume across AZ
 - Take a snapshot
 - Restore the snapshot to another AZ
 - EBS backups use IO and you shouldn't run them while your application is handling a lot of traffic
- Root EBS Volumes of instances get terminated by default if the EC2 instance gets terminated. (you can disable that)



EBS vs EFS— Elastic File System

- Mounting 100s of instances across AZ
- EFS share website files (WordPress)
- Only for Linux Instances (POSIX)
- EFS has a higher price point than EBS
- Can leverage Storage Tiers for cost savings
- Remember: EFS vs EBS vs Instance Store

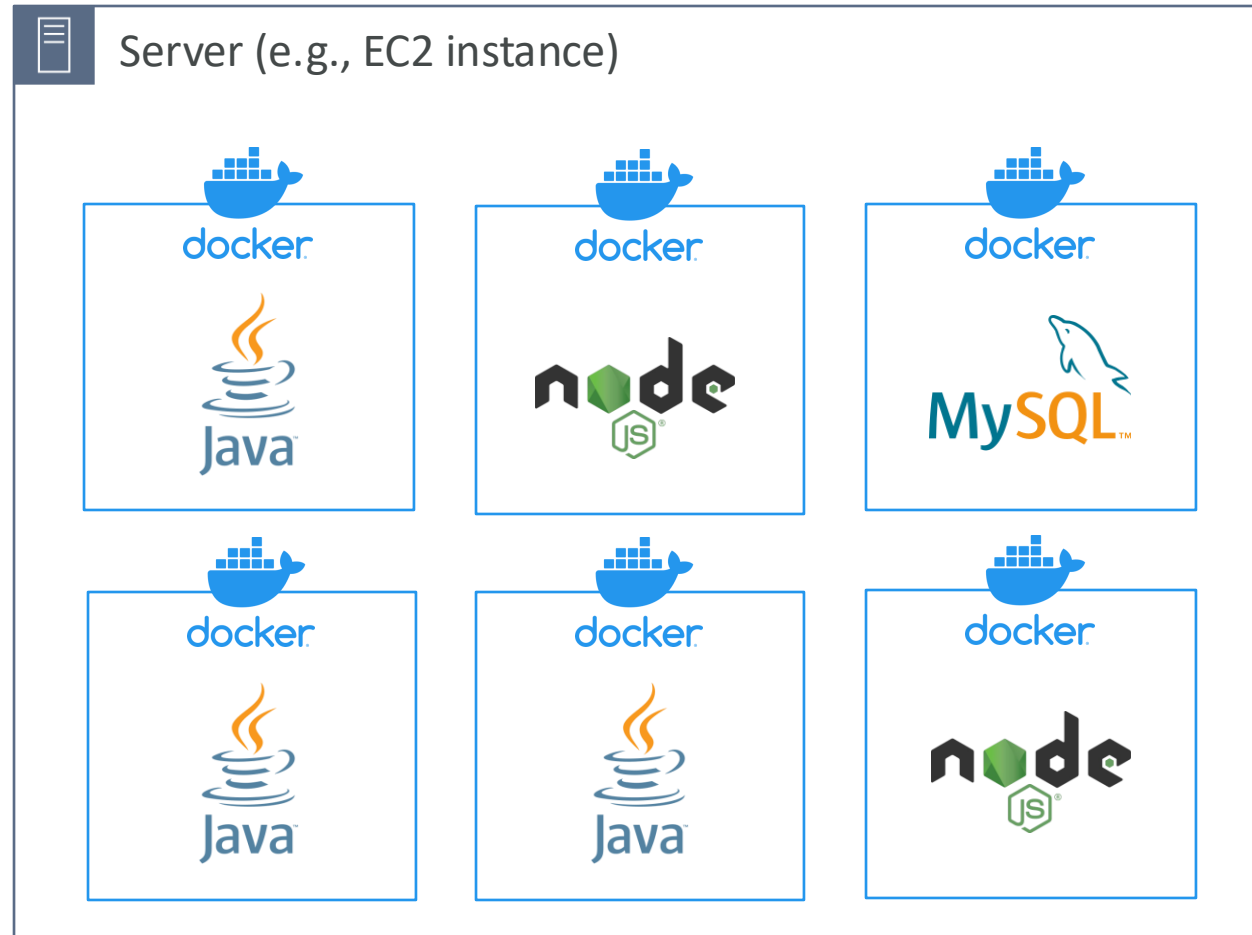




What is Docker?

- Docker is a software development platform to deploy apps
- Apps are packaged in containers that can be run on any OS
- Apps run the same, regardless of where they're run
 - Any machine
 - No compatibility issues
 - Predictable behavior
 - Less work
 - Easier to maintain and deploy
 - Works with any language, any OS, any technology
- Use cases: microservices architecture, lift-and-shift apps from on-premises to the AWS cloud, ...

Docker on an OS

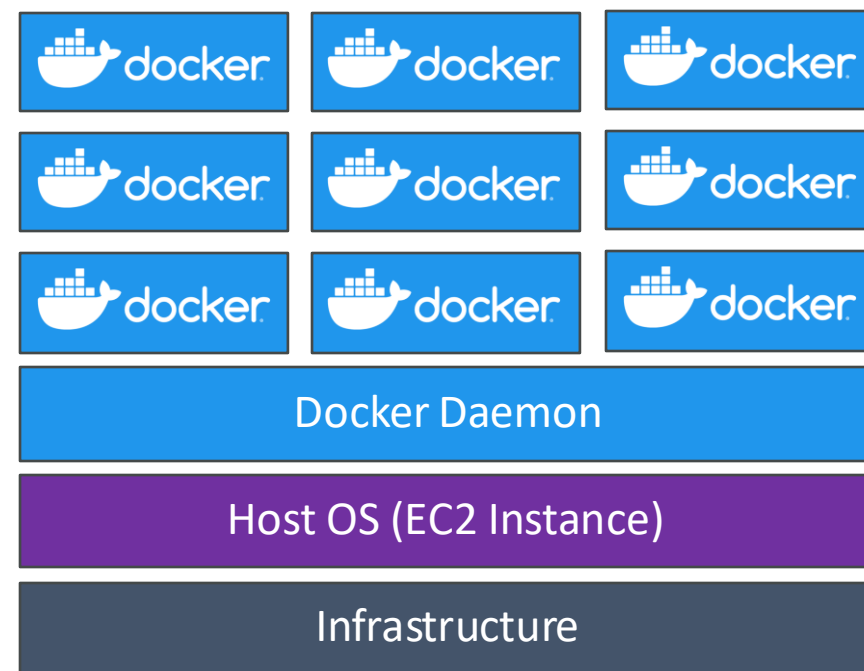
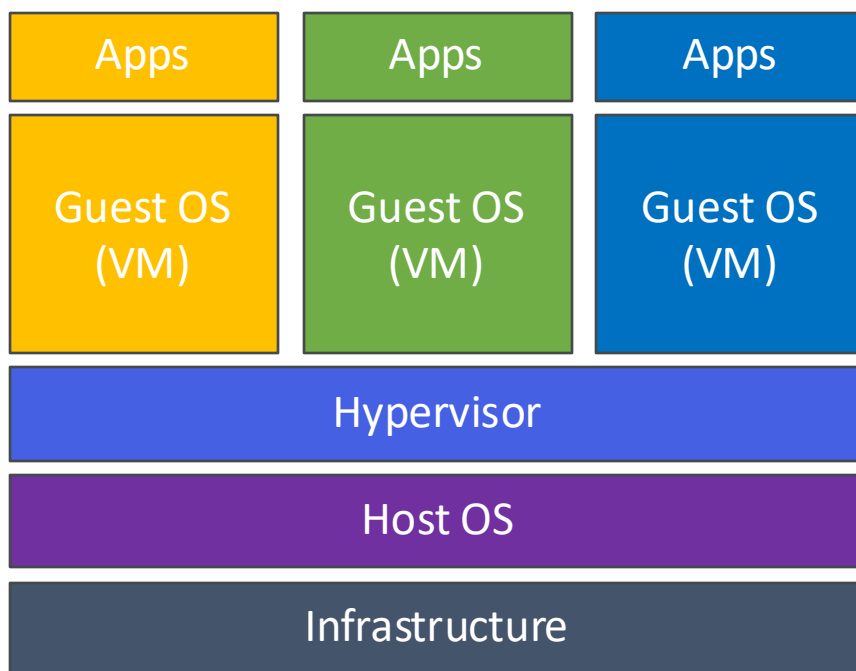


Where are Docker images stored?

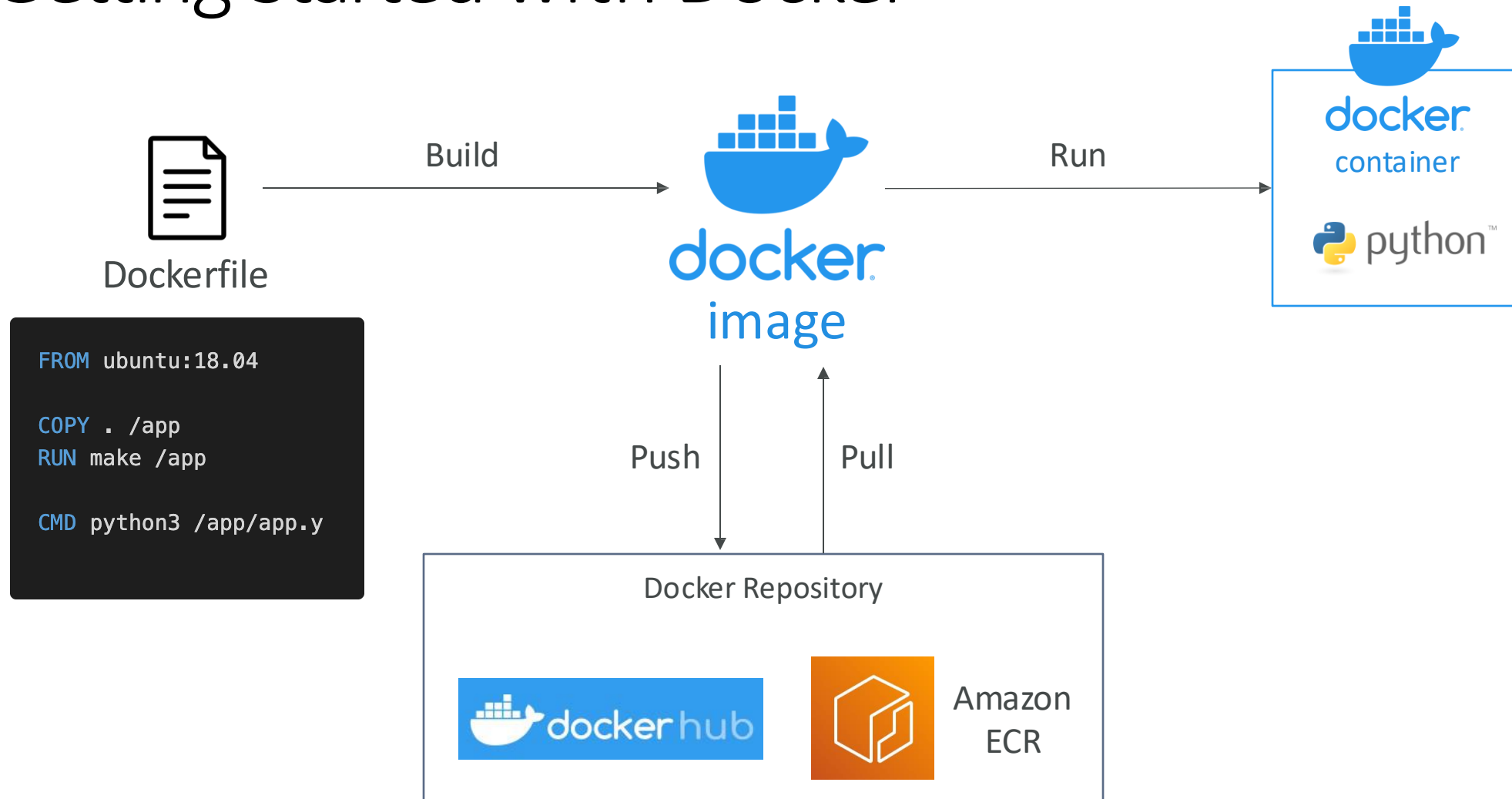
- Docker images are stored in Docker Repositories
- Docker Hub (<https://hub.docker.com>)
 - Public repository
 - Find base images for many technologies or OS (e.g., Ubuntu, MySQL, ...)
- Amazon ECR (Amazon Elastic Container Registry)
 - Private repository
 - Public repository (Amazon ECR Public Gallery <https://gallery.ecr.aws>)

Docker vs. Virtual Machines

- Docker is "sort of" a virtualization technology, but not exactly
- Resources are shared with the host => many containers on one server



Getting Started with Docker



Docker Containers Management on AWS

- Amazon Elastic Container Service (Amazon ECS)
 - Amazon's own container platform
- Amazon Elastic Kubernetes Service (Amazon EKS)
 - Amazon's managed Kubernetes (open source)
- AWS Fargate
 - Amazon's own Serverless container platform
 - Works with ECS and with EKS
- Amazon ECR:
 - Store container images



Amazon ECS



Amazon EKS



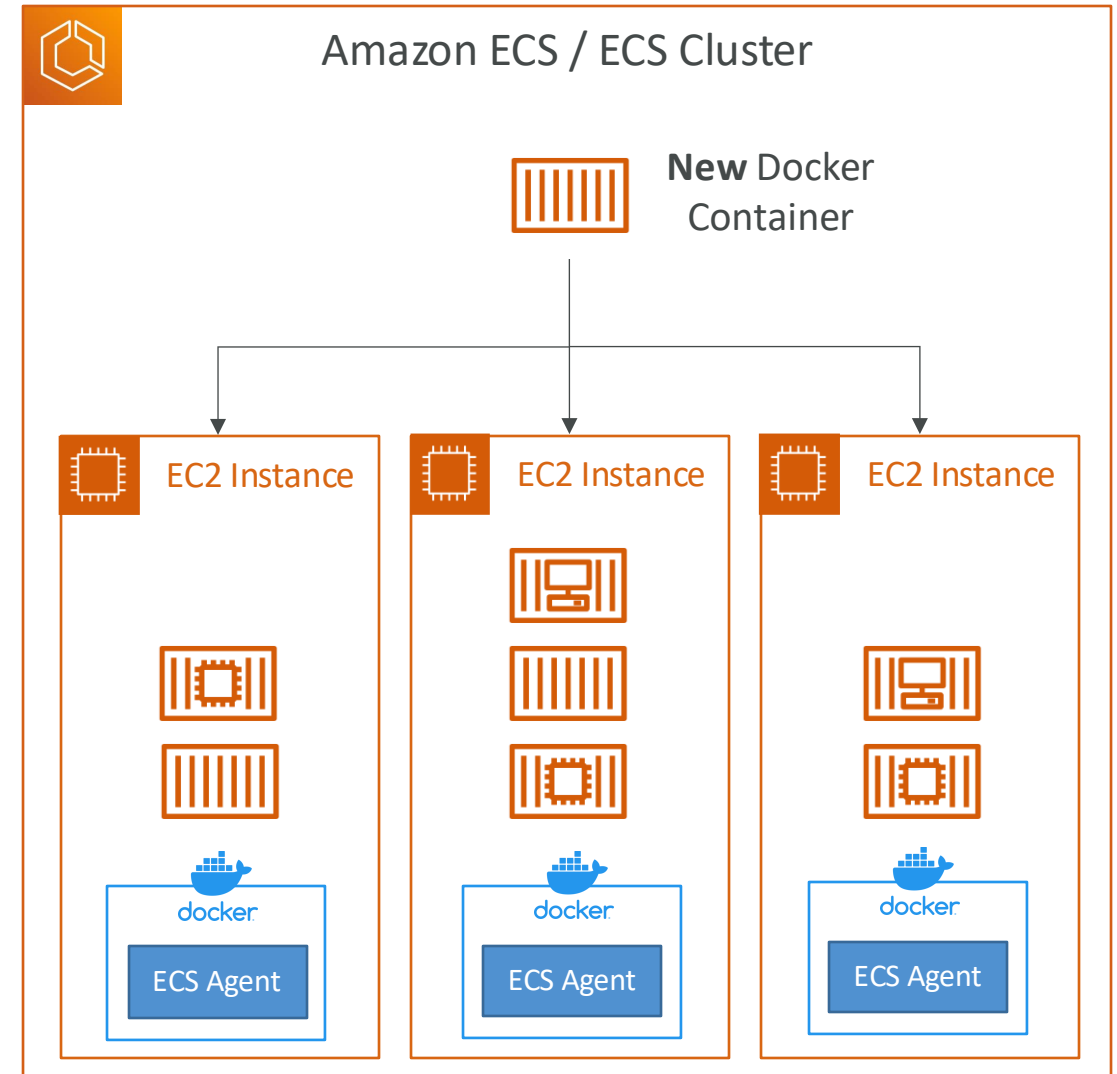
AWS Fargate



Amazon ECR

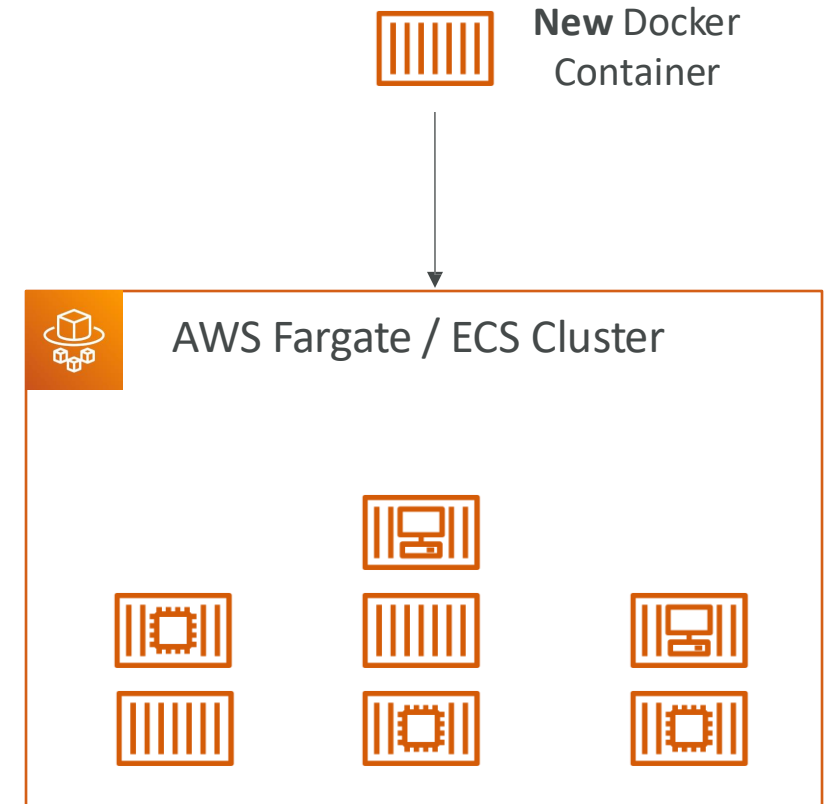
Amazon ECS - EC2 Launch Type

- ECS = Elastic Container Service
- Launch Docker containers on AWS = Launch ECSTasks on ECS Clusters
- EC2 Launch Type: you must provision & maintain the infrastructure (the EC2 instances)
- Each EC2 Instance must run the ECS Agent to register in the ECS Cluster
- AWS takes care of starting / stopping containers



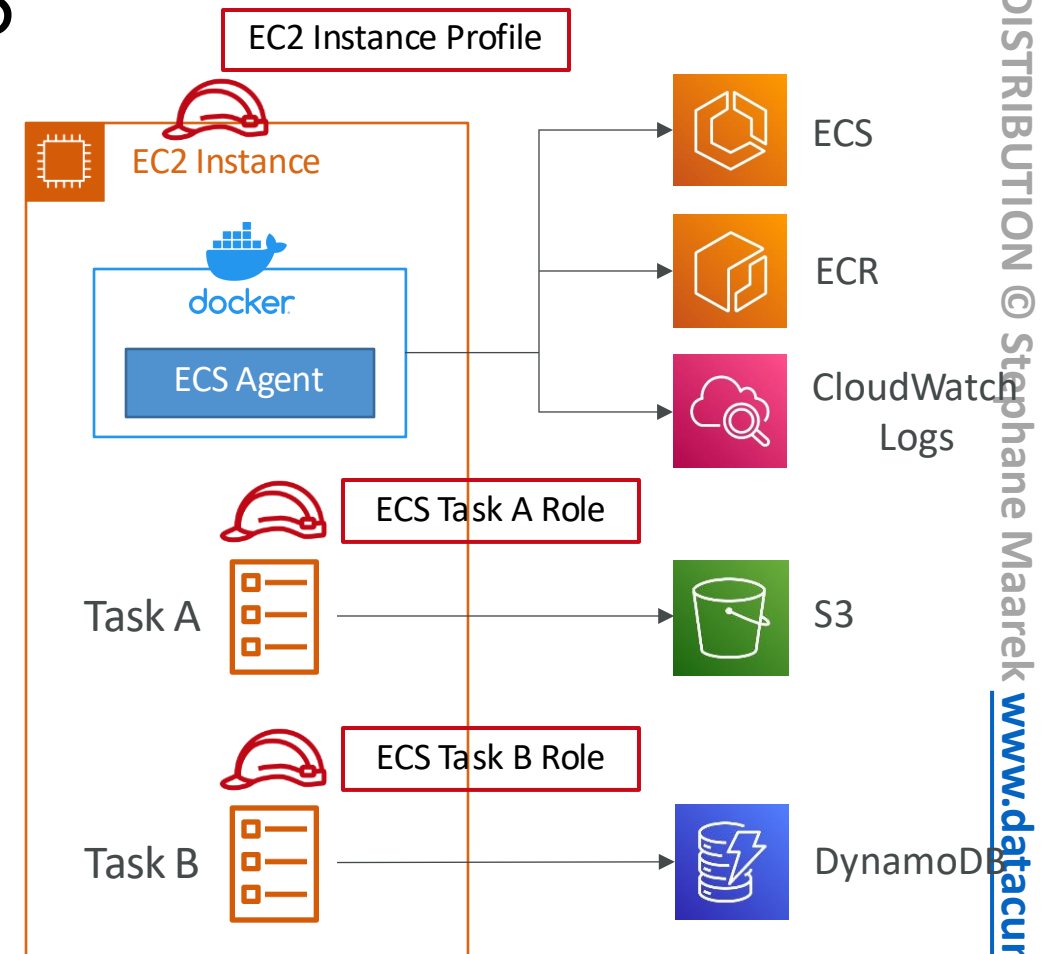
Amazon ECS – Fargate Launch Type

- Launch Docker containers on AWS
- You do not provision the infrastructure (no EC2 instances to manage)
- It's all Serverless!
- You just create task definitions
- AWS just runs ECSTasks for you based on the CPU / RAM you need
- To scale, just increase the number of tasks. Simple - no more EC2 instances



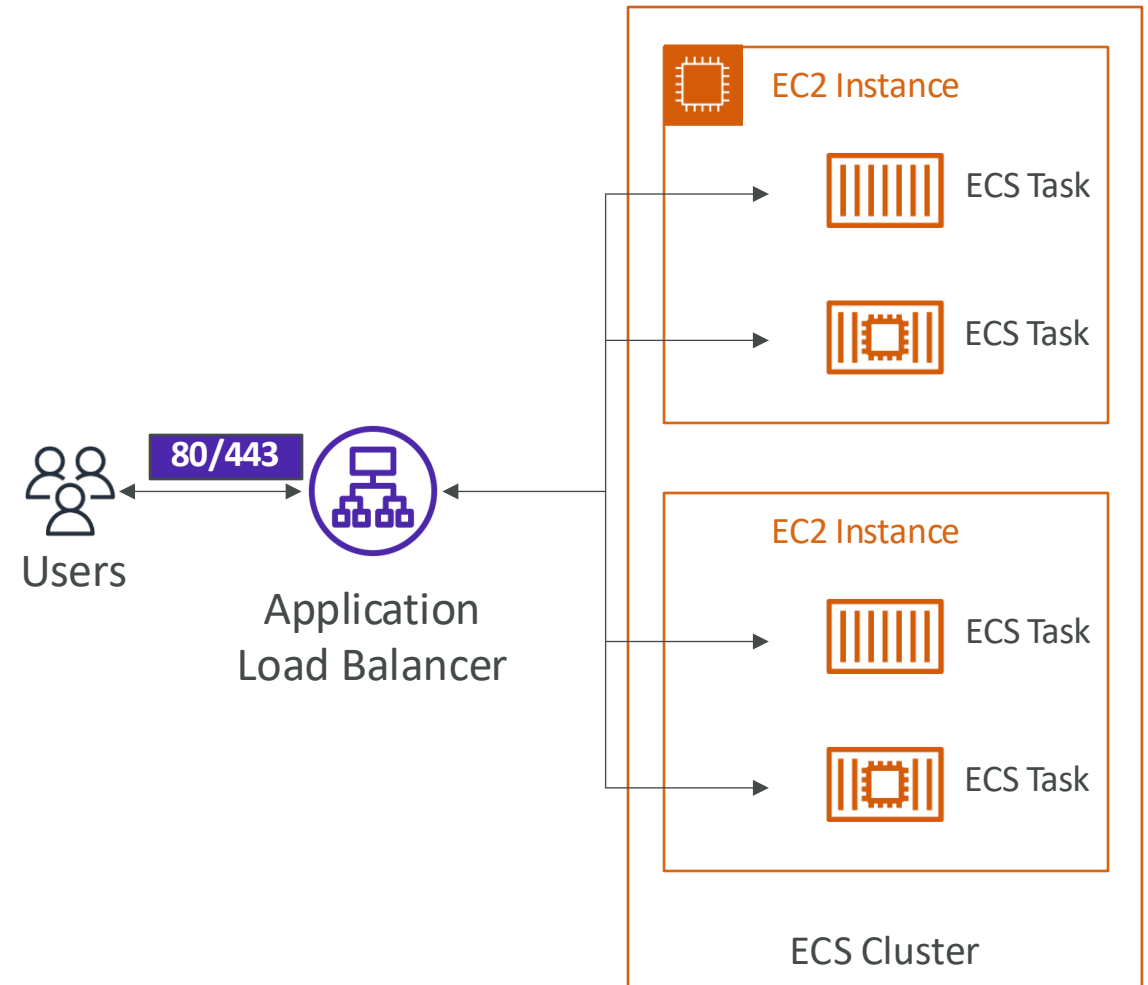
Amazon ECS – IAM Roles for ECS

- EC2 Instance Profile (EC2 Launch Type only):
 - Used by the ECS agent
 - Makes API calls to ECS service
 - Send container logs to CloudWatch Logs
 - Pull Docker image from ECR
 - Reference sensitive data in Secrets Manager or SSM Parameter Store
- ECSTask Role:
 - Allows each task to have a specific role
 - Use different roles for the different ECS Services you run
 - Task Role is defined in the task definition



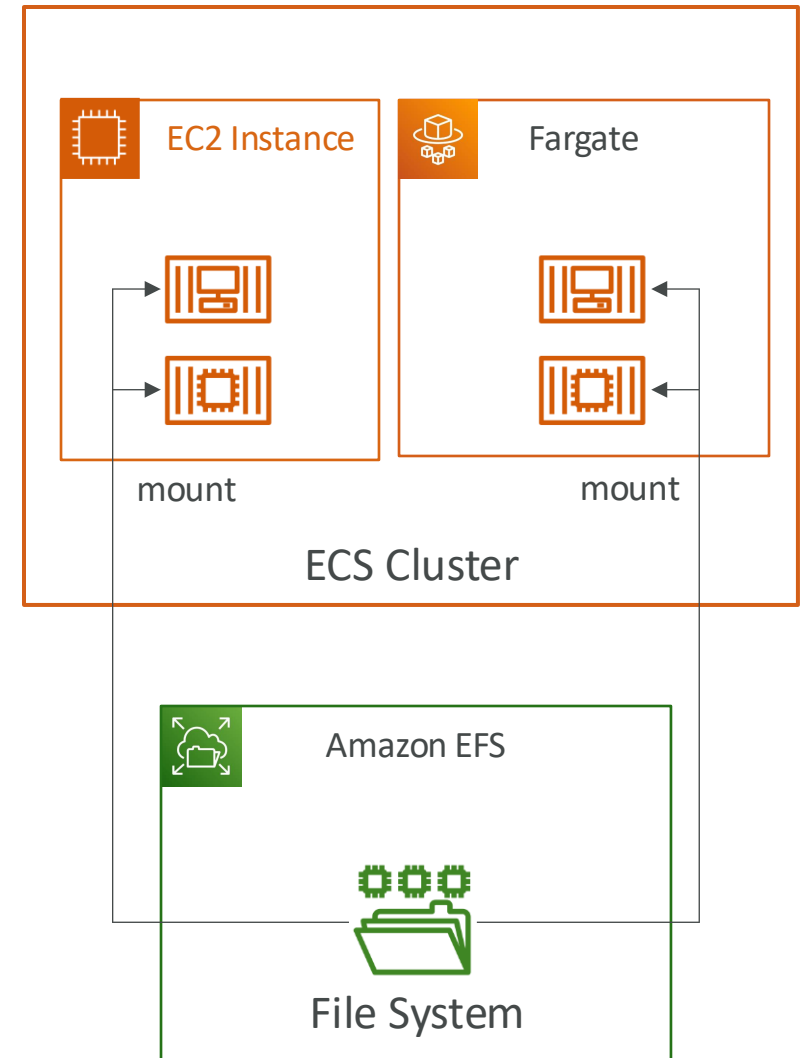
Amazon ECS – Load Balancer Integrations

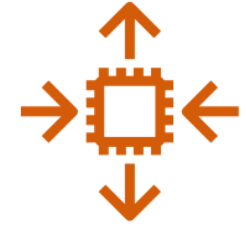
- **Application Load Balancer** supported and works for most use cases
- **Network Load Balancer** recommended only for high throughput / high performance use cases, or to pair it with AWS Private Link
- Classic Load Balancer supported but not recommended (no advanced features - no Fargate)



Amazon ECS – Data Volumes (EFS)

- Mount EFS file systems onto ECS tasks
- Works for both EC2 and Fargate launch types
- Tasks running in any AZ will share the same data in the EFS file system
- Fargate + EFS = Serverless
- Use cases: persistent multi-AZ shared storage for your containers
- Note:
 - Amazon S3 cannot be mounted as a file system





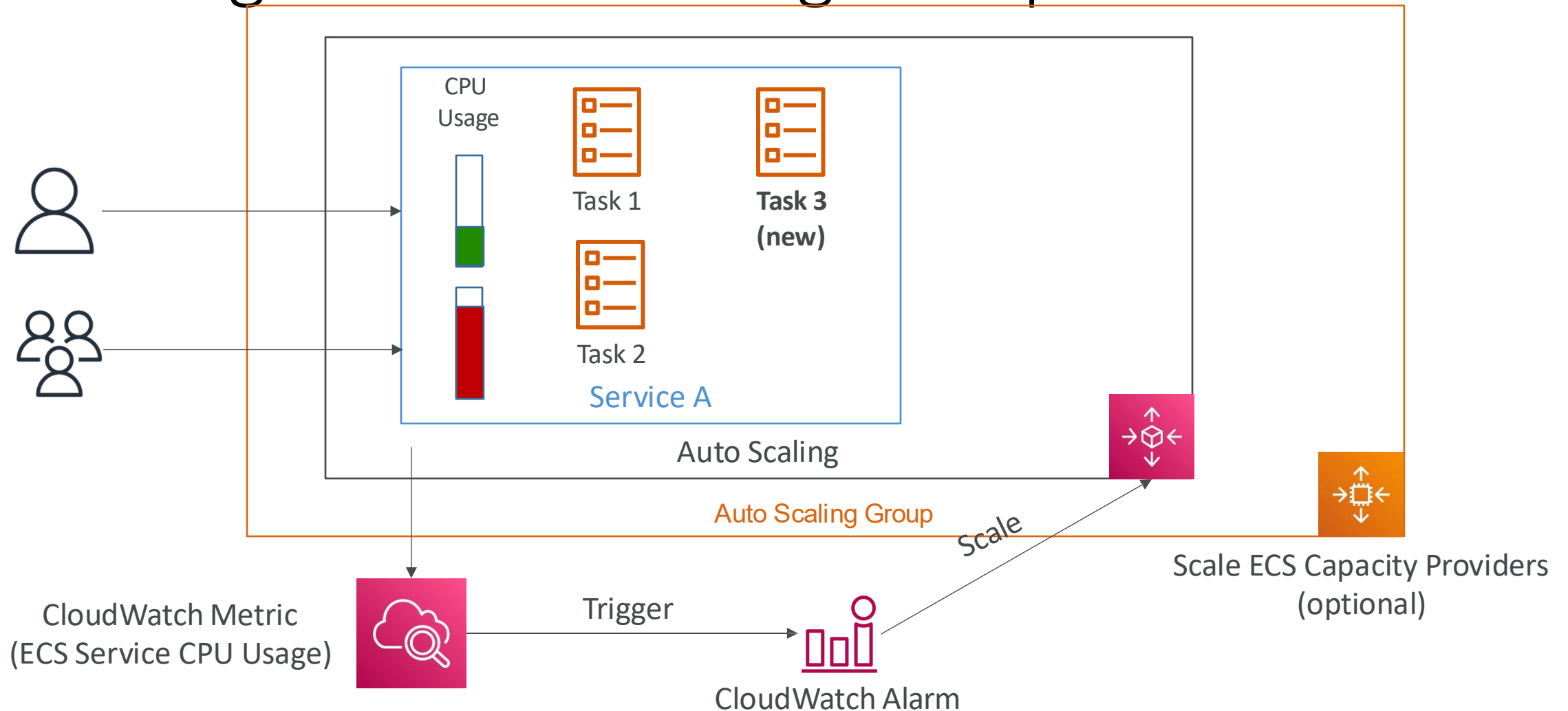
ECS Service Auto Scaling

- Automatically increase/decrease the desired number of ECS tasks
- Amazon ECS Auto Scaling uses AWS Application Auto Scaling
 - ECS Service Average CPU Utilization
 - ECS Service Average Memory Utilization - Scale on RAM
 - ALB Request Count PerTarget - metric coming from the ALB
- Target Tracking - scale based on target value for a specific CloudWatch metric
- Step Scaling - scale based on a specified CloudWatch Alarm
- Scheduled Scaling - scale based on a specified date/time (predictable changes)
- ECS Service Auto Scaling (task level) $\neq$ EC2 Auto Scaling (EC2 instance level)
- Fargate Auto Scaling is much easier to setup (because Serverless)

EC2 Launch Type – Auto Scaling EC2 Instances

- Accommodate ECS Service Scaling by adding underlying EC2 Instances
- Auto Scaling Group Scaling
 - Scale your ASG based on CPU Utilization
 - Add EC2 instances over time
- ECS Cluster Capacity Provider
 - Used to automatically provision and scale the infrastructure for your ECSTasks
 - Capacity Provider paired with an Auto Scaling Group
 - Add EC2 Instances when you're missing capacity (CPU, RAM...)

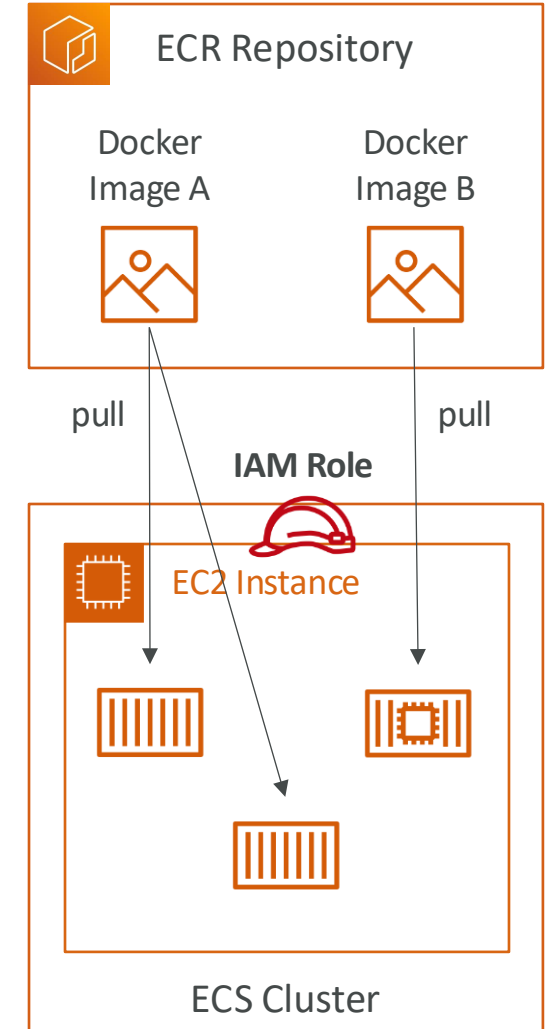
ECS Scaling – Service CPU Usage Example





Amazon ECR

- ECR = Elastic Container Registry
- Store and manage Docker images on AWS
- Private and Public repository (Amazon ECR Public Gallery <https://gallery.ecr.aws>)
- Fully integrated with ECS, backed by Amazon S3
- Access is controlled through IAM (permission errors => policy)
- Supports image vulnerability scanning, versioning, image tags, image lifecycle, ...

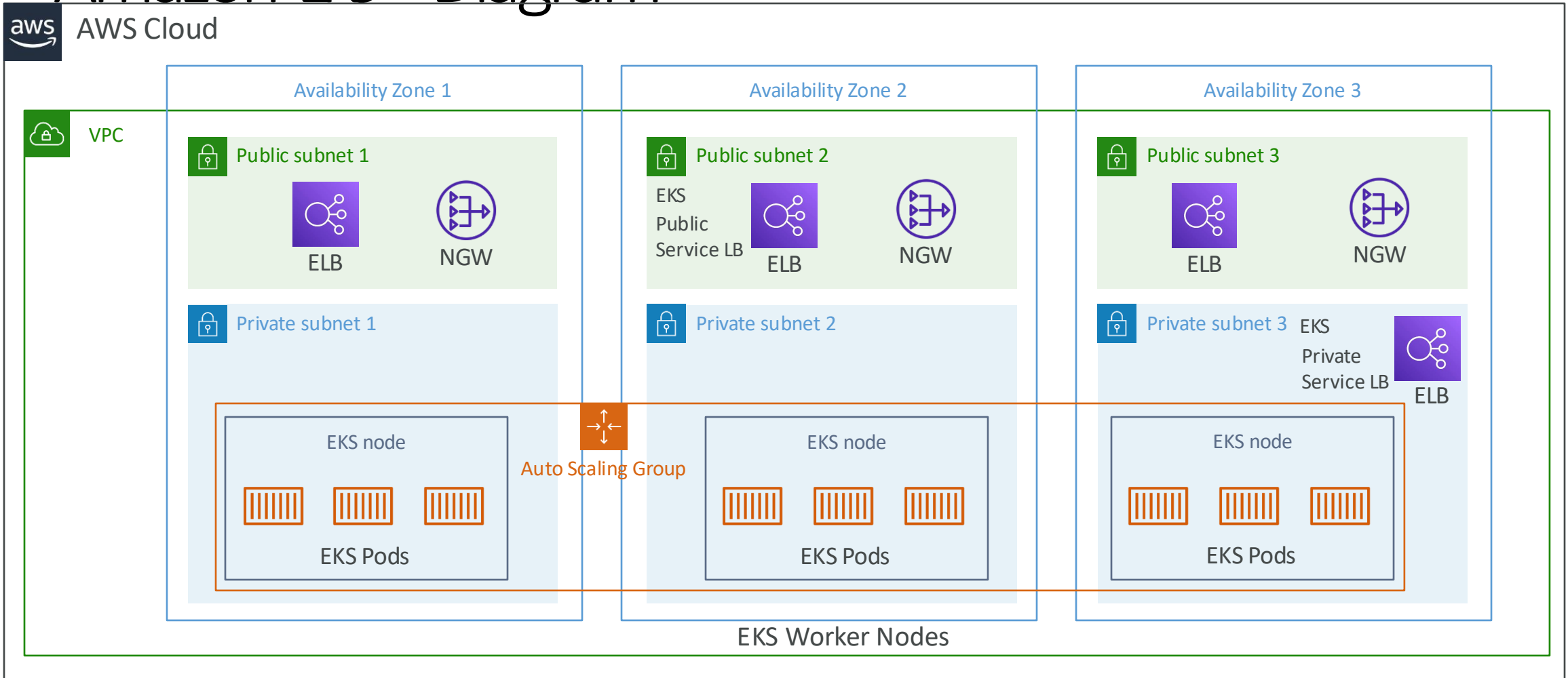




Amazon EKS Overview

- Amazon EKS = Amazon Elastic Kubernetes Service
- It is a way to launch managed Kubernetes clusters on AWS
- Kubernetes is an open-source system for automatic deployment, scaling and management of containerized (usually Docker) application
- It's an alternative to ECS, similar goal but different API
- EKS supports EC2 if you want to deploy worker nodes or Fargate to deploy serverless containers
- Use case: if your company is already using Kubernetes on-premises or in another cloud, and wants to migrate to AWS using Kubernetes
- Kubernetes is cloud-agnostic (can be used in any cloud - Azure, GCP...)
- For multiple regions, deploy one EKS cluster per region
- Collect logs and metrics using CloudWatch Container Insights

Amazon EKS - Diagram



Amazon EKS— Node Types

- Managed Node Groups
 - Creates and manages Nodes (EC2 instances) for you
 - Nodes are part of an ASG managed by EKS
 - Supports On-Demand or Spot Instances
- Self-Managed Nodes
 - Nodes created by you and registered to the EKS cluster and managed by an ASG
 - You can use prebuilt AMI - Amazon EKS Optimized AMI
 - Supports On-Demand or Spot Instances
- AWS Fargate
 - No maintenance required; no nodes managed

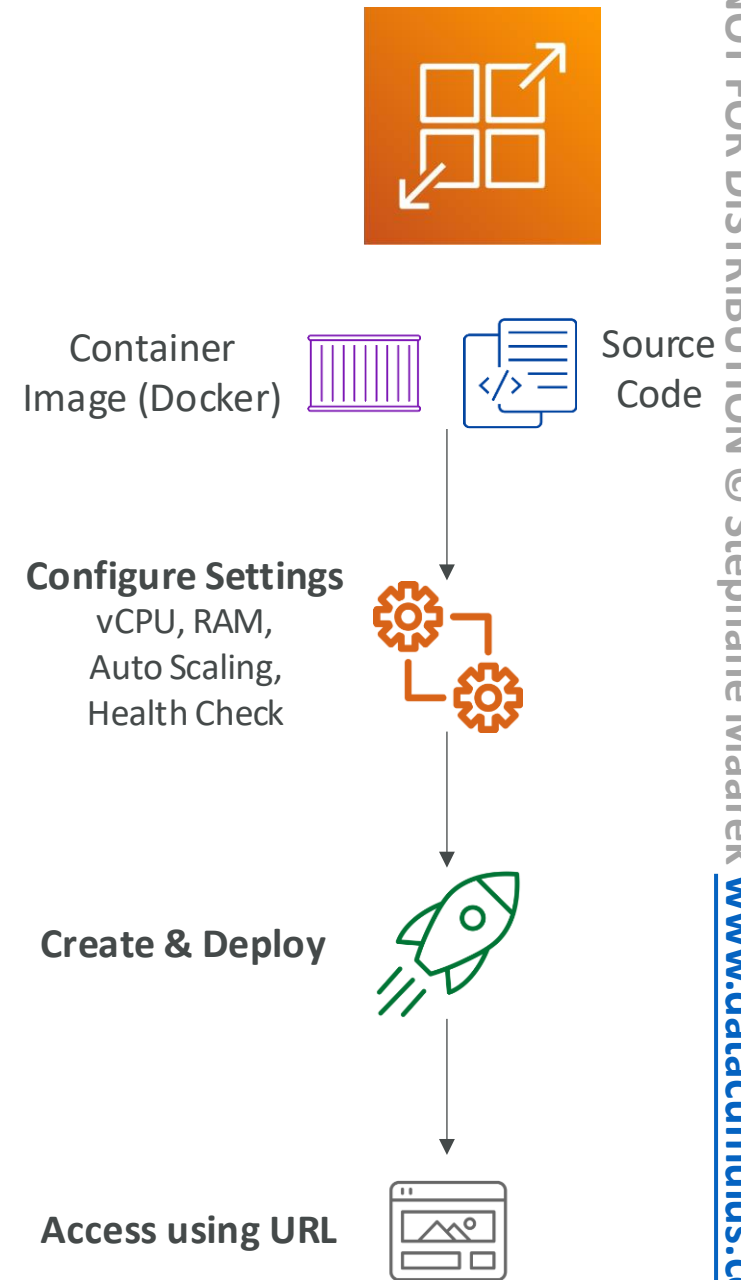
Amazon EKS – Data Volumes

- Need to specify StorageClass manifest on your EKS cluster
- Leverages a Container Storage Interface (CSI) compliant driver
- Support for...
- Amazon EBS
- Amazon EFS (works with Fargate)
- Amazon FSx for Lustre
- Amazon FSx for NetApp ONTAP



AWS App Runner

- Fully managed service that makes it easy to deploy web applications and APIs at scale
- No infrastructure experience required
- Start with your source code or container image
- Automatically builds and deploy the web app
- Automatic scaling, highly available, load balancer, encryption
- VPC access support
- Connect to database, cache, and message queue services
- Use cases: web apps, APIs, microservices, rapid production deployments



AWS App2Container (A2C)

- CLI tool for migrating and modernizing Java and .NET web apps into Docker Containers
- Lift-and-shift your apps running in on-premises bare metal, virtual machines, or in any Cloud to AWS
- Accelerate modernization, no code changes, migrate legacy apps...
- Generates CloudFormation templates (compute, network...)
- Register generated Docker containers to ECR
- Deploy to ECS, EKS, or App Runner
- Supports pre-built CI/CD pipelines

AWS App2Container (A2C)

